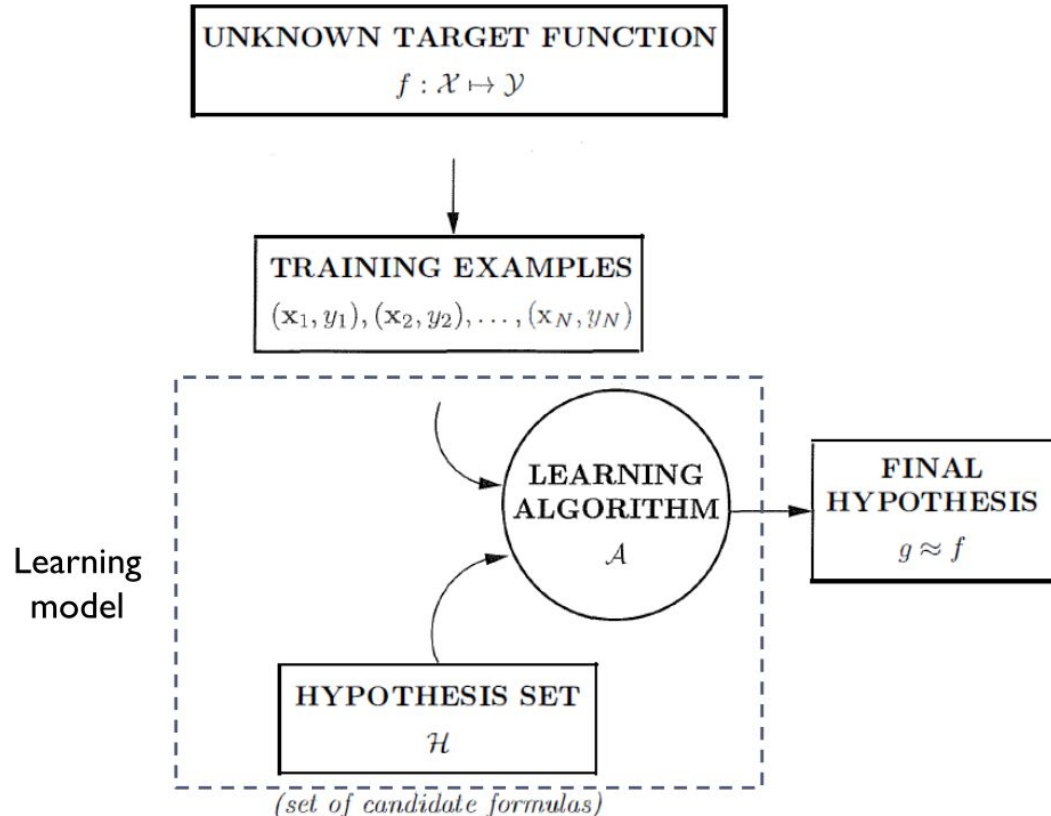


Linear Regression

Fatemeh Mansoori

Components of Supervised Learning



Main Steps of Learning Tasks

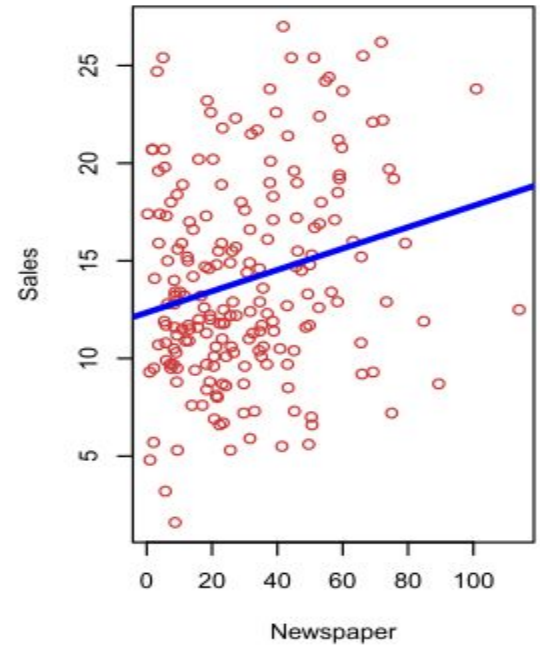
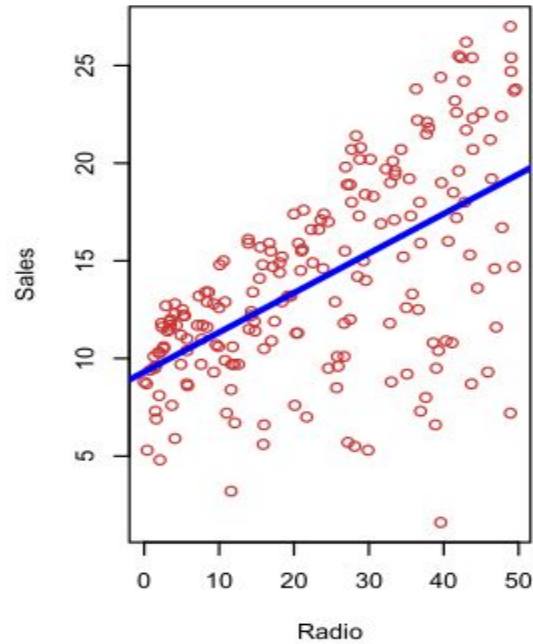
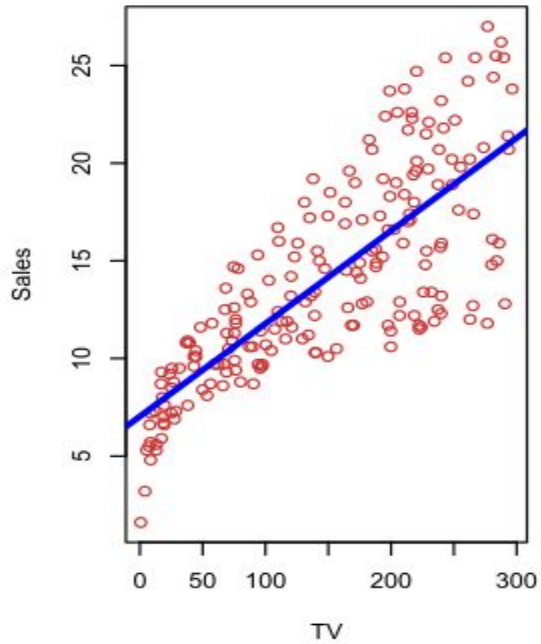
- Selection of hypothesis set (or model specification)
 - Which class of models (mappings) should we use for our data?
- Learning: find mapping f (from hypothesis set) based on the training data
 - Which notion of error should we use? (loss functions)
 - Optimization of loss function to find mapping f
- Evaluation: how well f generalizes to yet unseen examples
 - How do we ensure that the error on future data is minimized? (generalization)

A sample problem(Advertising)

- The Advertising data set.
 - sales, in thousands of units
 - TV, radio, and newspaper budgets, in thousands of dollars
 - 200 different markets.

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9

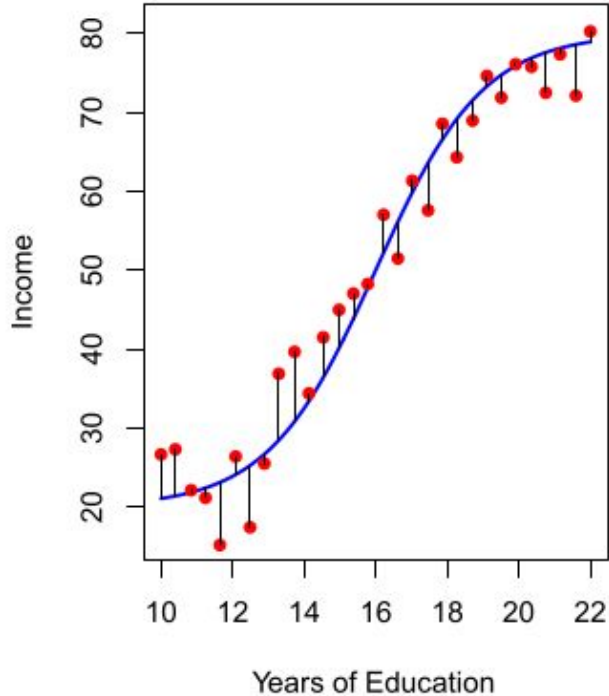
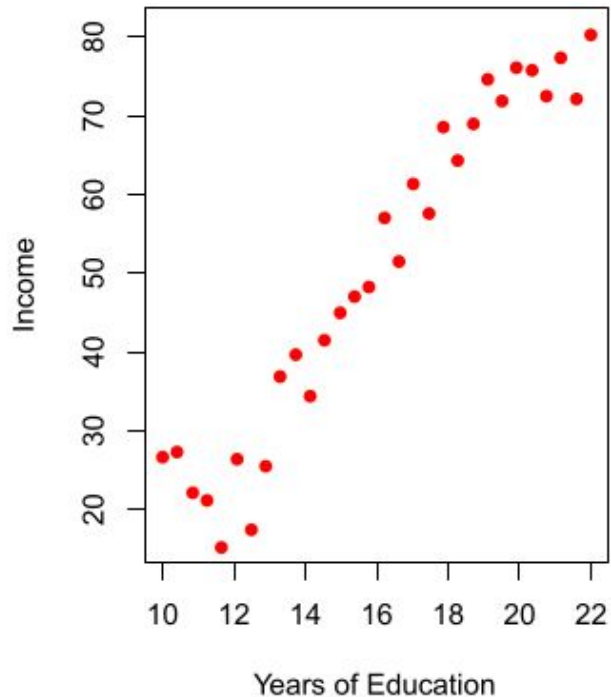
Linear function (Univariate)



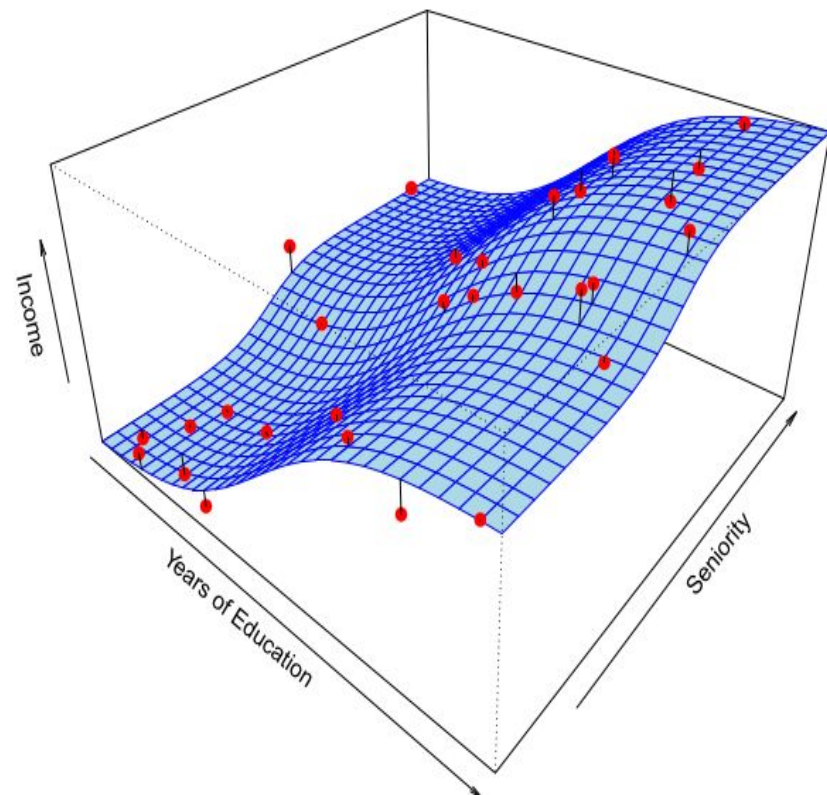
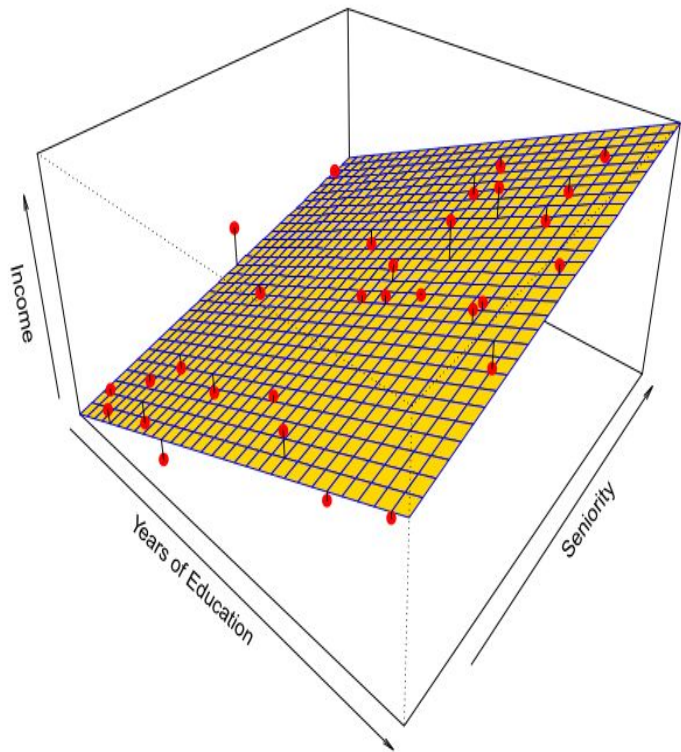
A sample problem (Income)

Observation	Years of Education	Years of Seniority	Annual Income
1	12	2	35.4
2	12	8	46.8
3	14	3	47.1
4	14	10	61.5
5	16	5	63.2
6	16	12	78.4

Non linear function (univariate)



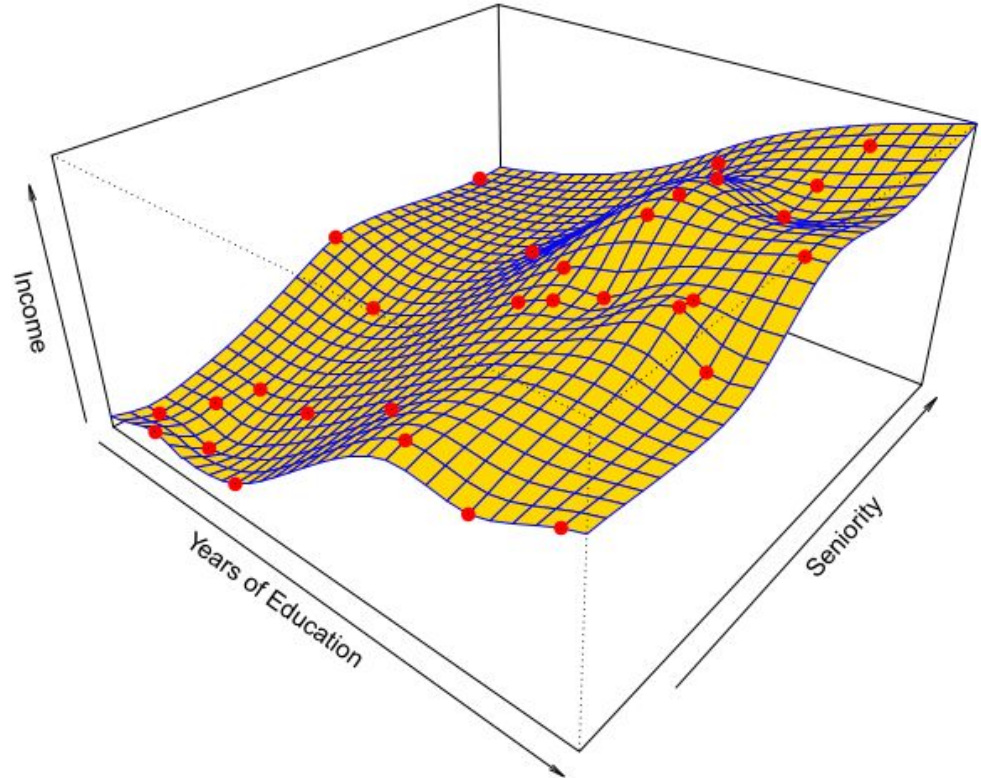
Multivariate



Multivariate

What is the difference of this function ?

Does this function better than the others?



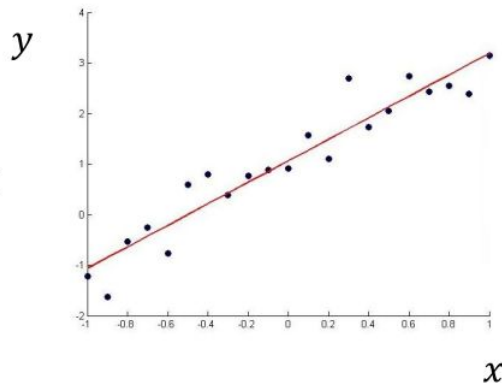
Hypothesis space

- Specify the class of functions (e.g linear)
- We begin by the class of linear functions
 - Easy to extend to no linear models

Linear Regression

Univariate

$$f : \mathbb{R} \rightarrow \mathbb{R} \quad f(x; \mathbf{w}) = w_0 + w_1 x$$



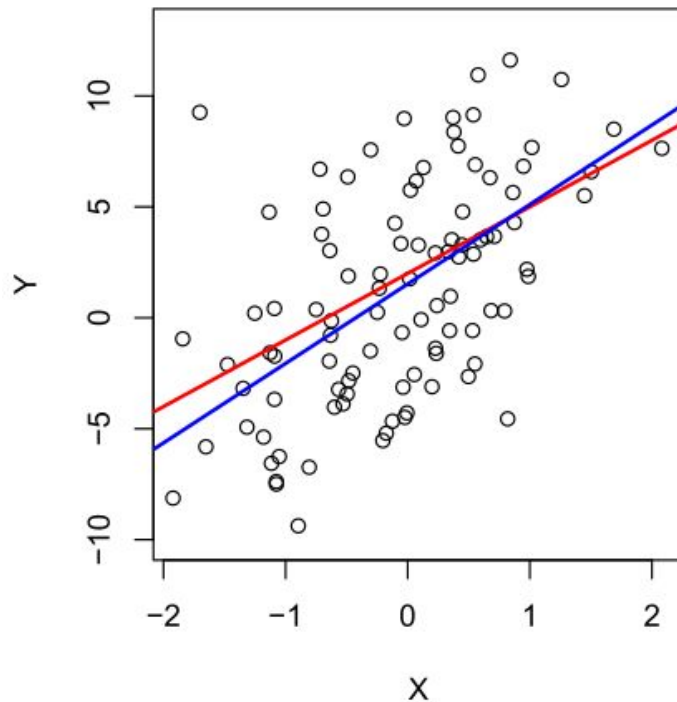
Multivariate

$$f : \mathbb{R}^d \rightarrow \mathbb{R} \quad f(\mathbf{x}; \mathbf{w}) = w_0 + w_1 x_1 + \dots w_d x_d$$

$\mathbf{w} = [w_0, w_1, \dots, w_d]^T$ are parameters we need to set.

Linear regression (univariate)

- Linear univariate function : $f(x) = w_0 + w_1x$
- Different parameter $\rightarrow$ Different line
- Which line is better ?

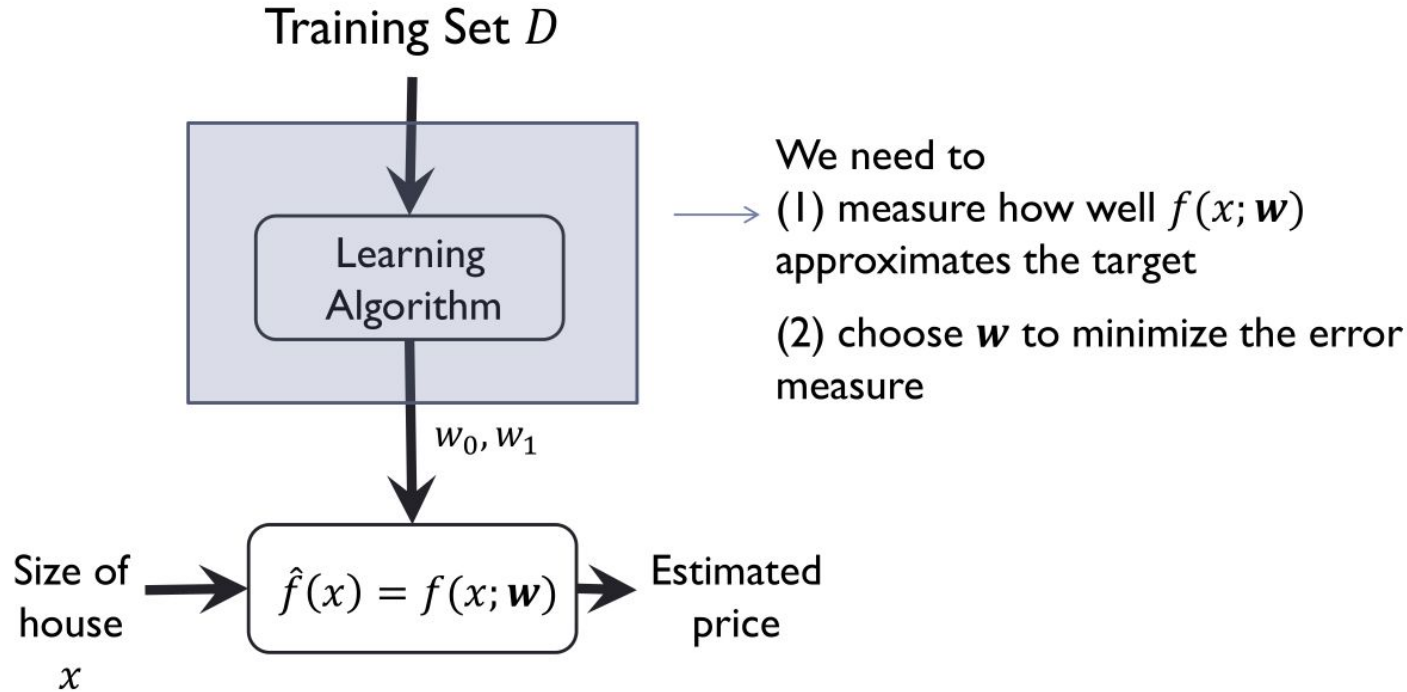


Learning problem

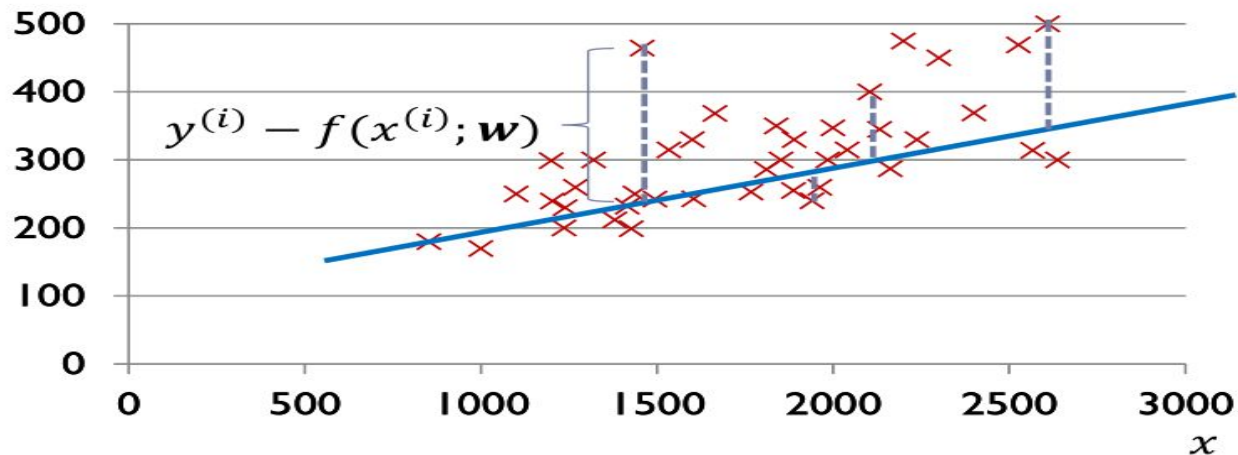
Learning (estimation): optimization of a cost function

- ▶ Based on the training set $D = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$ and a cost function we find (an estimate) $f \in F$ of the target function
- Select how to measure the error (i.e. prediction loss)
-
- Find the minimum of the resulting error or cost function

Learning Algorithm

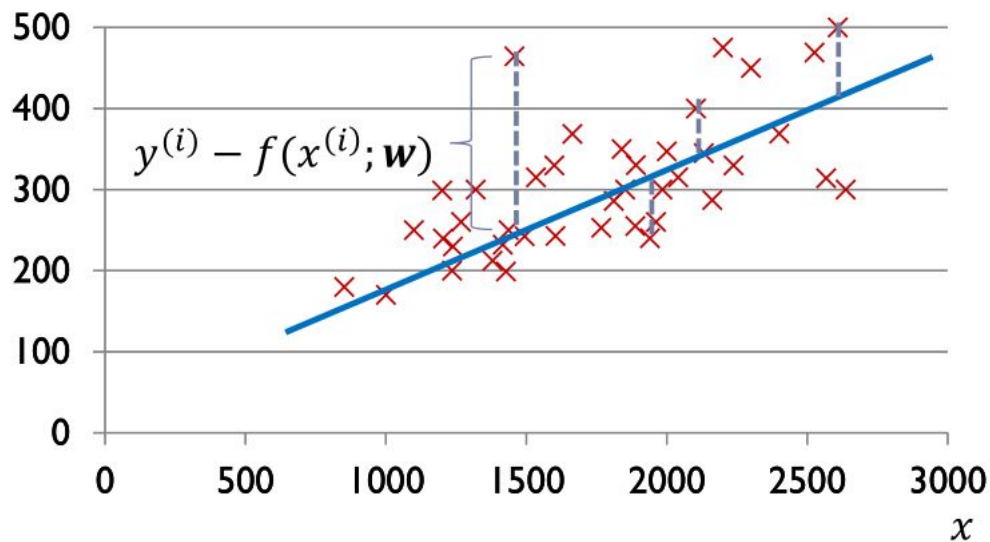


How to measure the error



Squared error: $\left(y^{(i)} - f(x^{(i)}; \mathbf{w})\right)^2$

Linear regression: univariate example



Cost function:

$$\begin{aligned} J(\mathbf{w}) &= \sum_{i=1}^n (y^{(i)} - f(x; \mathbf{w}))^2 \\ &= \sum_{i=1}^n (y^{(i)} - w_0 - w_1 x^{(i)})^2 \end{aligned}$$

Regression: squared loss

- ▶ In the SSE cost function, we used squared error as the prediction loss:

$$Loss(y, \hat{y}) = (y - \hat{y})^2 \qquad \hat{y} = f(\mathbf{x}; \mathbf{w})$$

- ▶ Cost function (based on the training set):

$$\begin{aligned} J(\mathbf{w}) &= \sum_{i=1}^n Loss\left(y^{(i)}, f(\mathbf{x}^{(i)}; \mathbf{w})\right) \\ &= \sum_{i=1}^n \left(y^{(i)} - f(\mathbf{x}^{(i)}; \mathbf{w})\right)^2 \end{aligned}$$

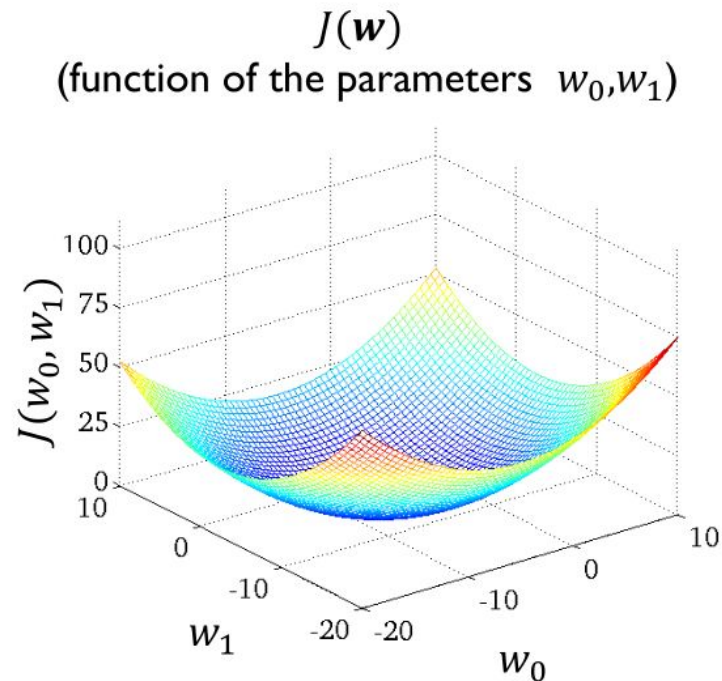
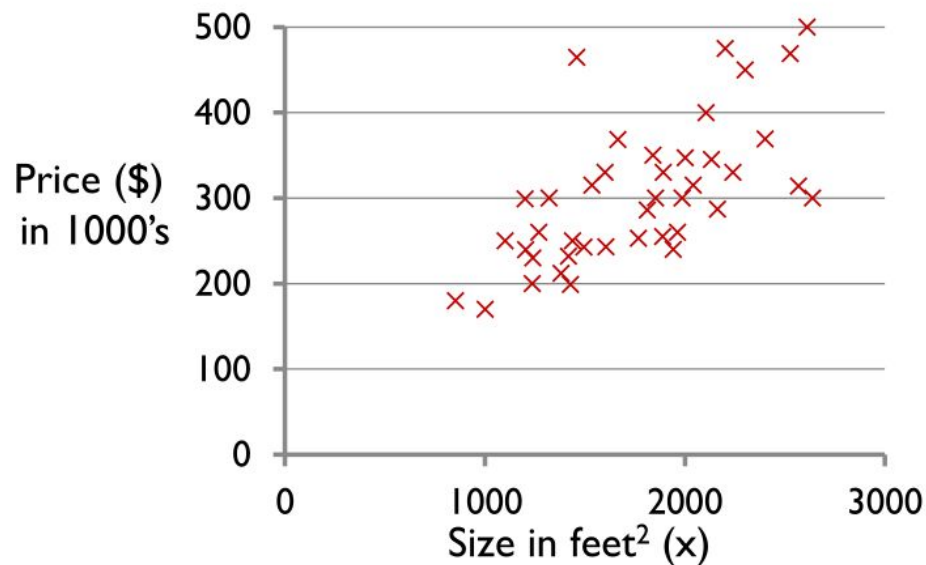
- ▶ Minimizing sum (or mean) of squared errors is a common approach in curve fitting, neural network, etc.

Sum of Squares Error (SSE) cost function

$$J(\mathbf{w}) = \sum_{i=1}^n (y^{(i)} - f(\mathbf{x}^{(i)}; \mathbf{w}))^2$$

- ▶ $J(\mathbf{w})$: sum of the squares of the prediction errors on the training set
- ▶ We want to find the best regression function $f(\mathbf{x}^{(i)}; \mathbf{w})$
 - ▶ equivalently, the best $\mathbf{w}$
- ▶ Minimize $J(\mathbf{w})$
 - ▶ Find optimal $\hat{f}(\mathbf{x}) = f(\mathbf{x}; \hat{\mathbf{w}})$ where $\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} J(\mathbf{w})$

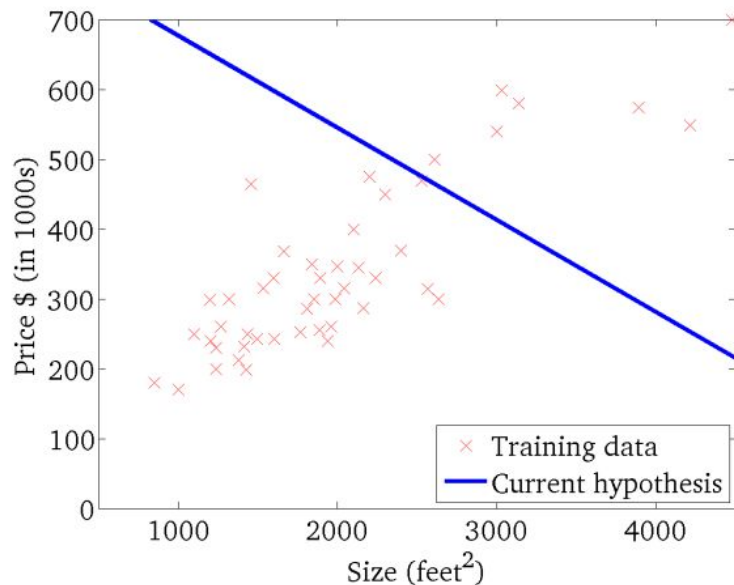
Cost function: univariate example



Cost function: univariate example

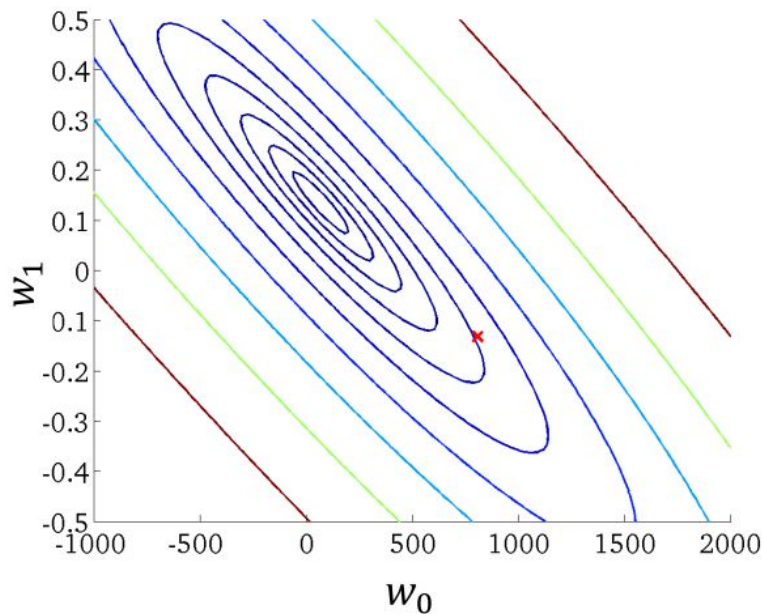
$$f(x; w_0, w_1) = w_0 + w_1 x$$

(for fixed w_0, w_1 , this is a function of x)



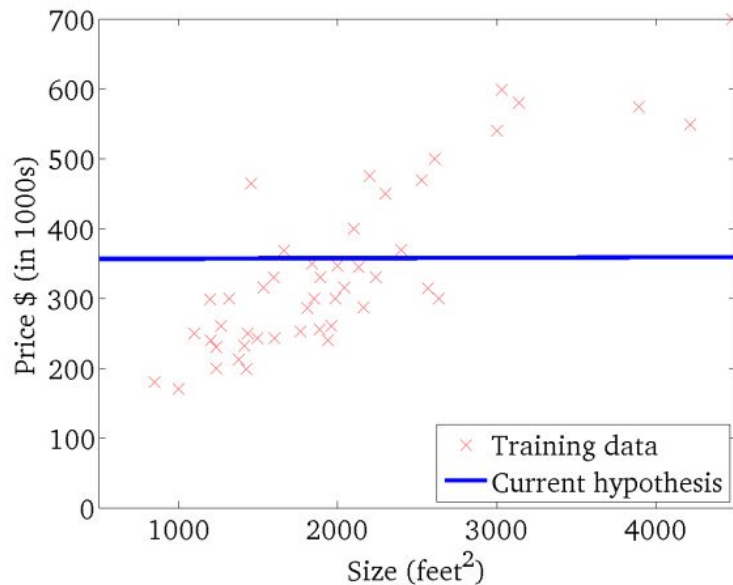
$$J(w_0, w_1)$$

(function of the parameters w_0, w_1)



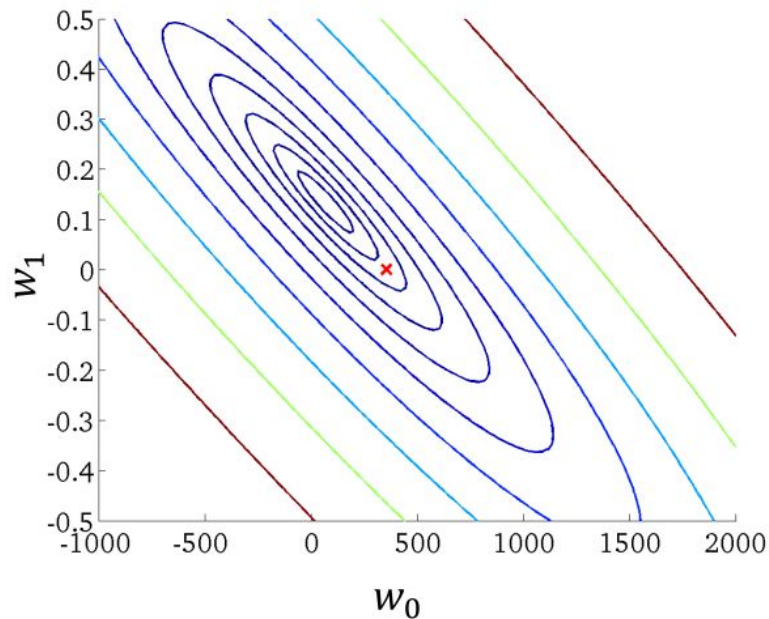
Cost function: univariate example

$$f(x; w_0, w_1) = w_0 + w_1 x$$



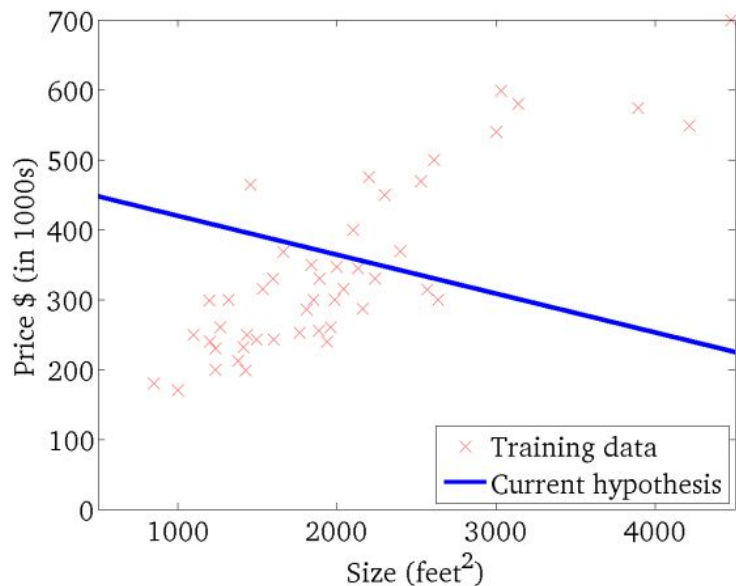
$$J(w_0, w_1)$$

(function of the parameters w_0, w_1)



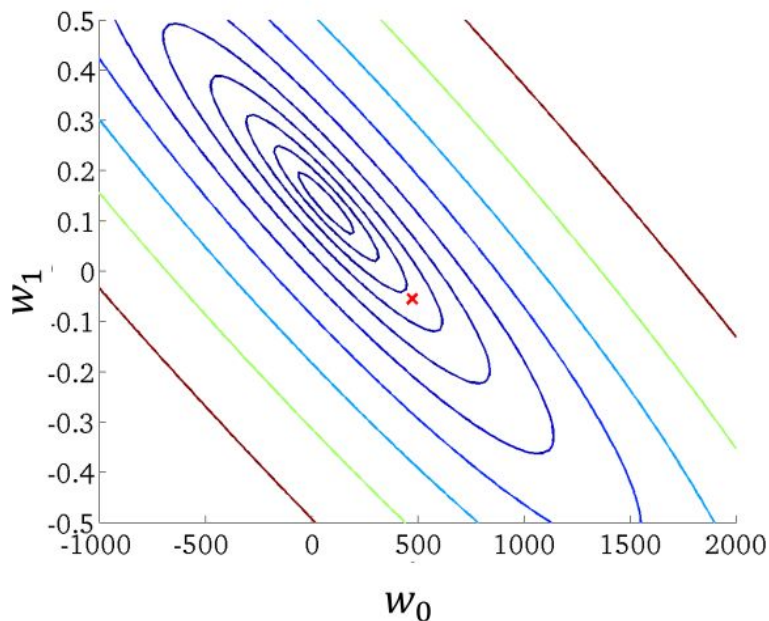
Cost function: univariate example

$$f(x; w_0, w_1) = w_0 + w_1 x$$



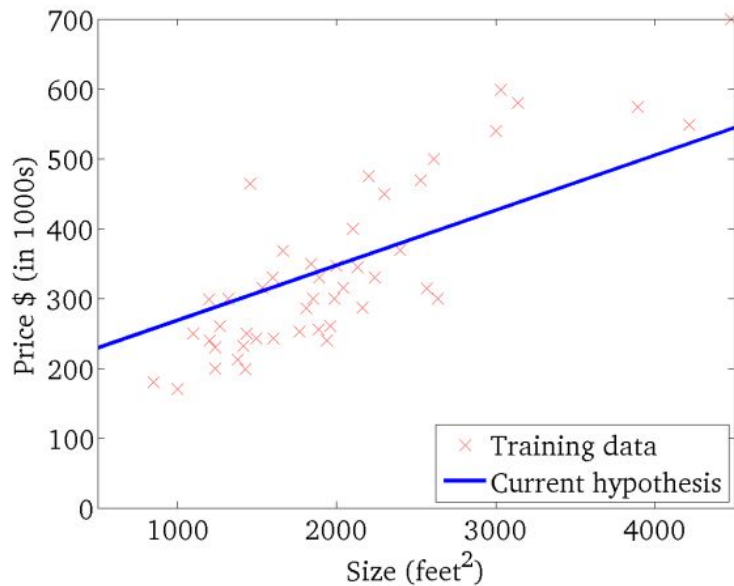
$$J(w_0, w_1)$$

(function of the parameters w_0, w_1)



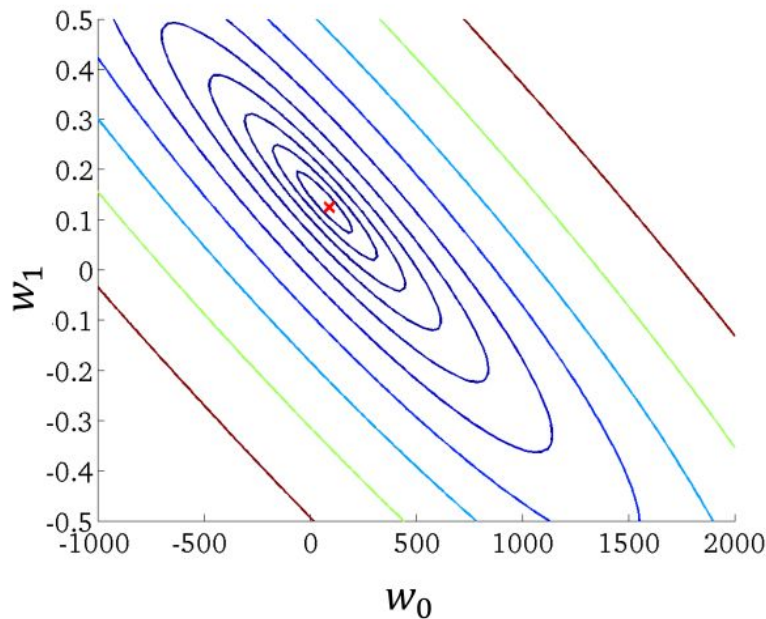
Cost function: univariate example

$$f(x; w_0, w_1) = w_0 + w_1 x$$



$$J(w_0, w_1)$$

(function of the parameters w_0, w_1)



Cost function optimization: univariate

$$J(\mathbf{w}) = \sum_{i=1}^n (y^{(i)} - w_0 - w_1 x^{(i)})^2$$

- Necessary conditions for the “optimal” parameter values:

$$\frac{\partial J(\mathbf{w})}{\partial w_0} = 0$$

$$\frac{\partial J(\mathbf{w})}{\partial w_1} = 0$$

Optimality conditions: univariate

$$J(\mathbf{w}) = \sum_{i=1}^n (y^{(i)} - w_0 - w_1 x^{(i)})^2$$

$$\frac{\partial J(\mathbf{w})}{\partial w_1} = \sum_{i=1}^n 2(y^{(i)} - w_0 - w_1 x^{(i)})(-x^{(i)}) = 0$$

$$\frac{\partial J(\mathbf{w})}{\partial w_0} = \sum_{i=1}^n 2(y^{(i)} - w_0 - w_1 x^{(i)})(-1) = 0$$

- A systems of 2 linear equations

Linear Regression Example

This is a real-world example of how businesses can use linear regression to make decisions about marketing budgets. The following table shows the amount of TV advertising budget spend and respective average sales of houses in Boston:

TV Advertising Spend (\$1000)	Sales (Units)
230.1	22.1
44.5	10.4
17.2	9.3
151.5	18.5
180.8	12.9

Table 1: TV Advertising Spend vs. Sales Dataset

Learn More: Advertising Sales Dataset

Linear Regression Example

- In this problem, Sales per TV advertising is need thus the amount spend is considered as input x and sales is considered as output y .
- Using linear regression, cost function can be written as:

$$J(w) = \sum_{i=1}^5 \left(y^{(i)} - w_0 - w_1 x^{(i)} \right)^2$$

- Applying necessary conditions for the optimal parameters:

$$\frac{\partial J(w)}{\partial w_1} = \sum_{i=1}^5 2 \left(y^{(i)} - w_0 - w_1 x^{(i)} \right) (-x^{(i)}) = 0 \implies 110863w_1 + 624.1w_0 - 10843.04 = 0$$

$$\frac{\partial J(w)}{\partial w_0} = \sum_{i=1}^5 2 \left(y^{(i)} - w_0 - w_1 x^{(i)} \right) (-1) = 0 \implies 624.1w_1 + 5w_0 - 73.2 = 0$$

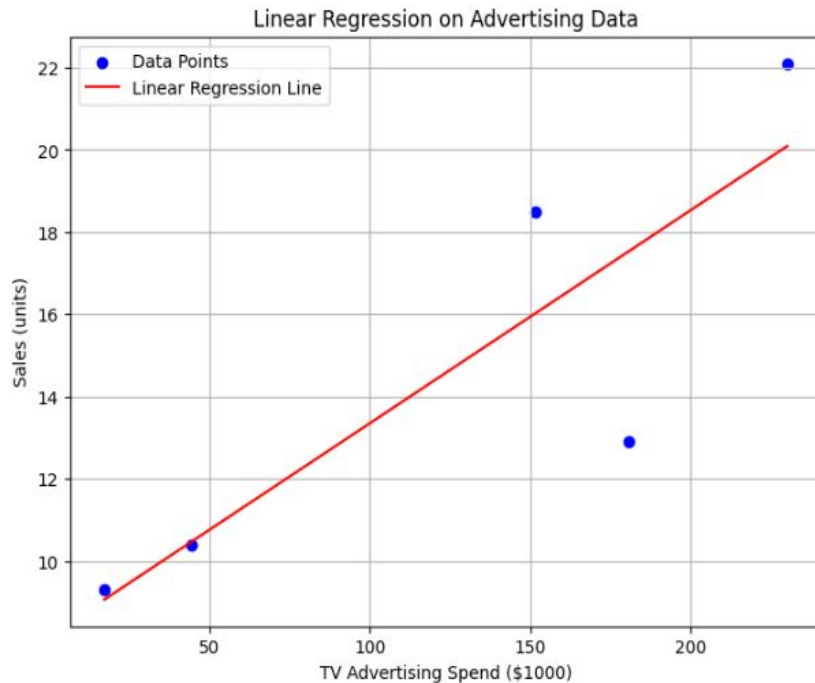
Linear Regression

- Solving the system of two equations described, we get:

$$w_1 \approx 0.052$$

$$w_0 \approx 8.18$$

TV Advertising Spend (\$1000)	Sales (Units)
230.1	22.1
44.5	10.4
17.2	9.3
151.5	18.5
180.8	12.9



Cost function: multivariate

- ▶ We have to minimize the empirical squared loss:

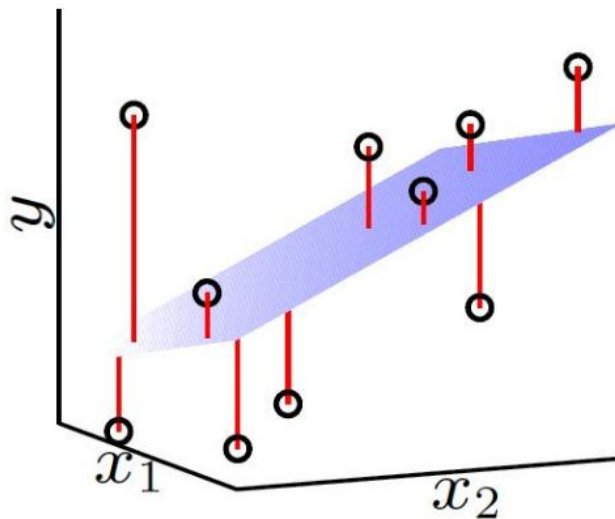
$$J(\mathbf{w}) = \sum_{i=1}^n \left(y^{(i)} - f(\mathbf{x}^{(i)}; \mathbf{w}) \right)^2$$

$$f(\mathbf{x}; \mathbf{w}) = w_0 + w_1 x_1 + \dots w_d x_d$$

$$\mathbf{w} = [w_0, w_1, \dots, w_d]^T$$

$$\hat{\mathbf{w}} = \operatorname{argmin}_{\mathbf{w} \in \mathbb{R}^{d+1}} J(\mathbf{w})$$

Cost function and optimal linear model



- ▶ Necessary conditions for the “optimal” parameter values:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \mathbf{0}$$

- ▶ A system of $d + 1$ linear equations

Cost function: matrix notation

$$\begin{aligned} J(\mathbf{w}) &= \sum_{i=1}^n \left(\mathbf{y}^{(i)} - f(\mathbf{x}^{(i)}; \mathbf{w}) \right)^2 = \\ &= \sum_{i=1}^n \left(\mathbf{y}^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)} \right)^2 \end{aligned}$$

$$\mathbf{y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \dots & x_d^{(n)} \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix}$$

$$J(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$$

Cost function: matrix notation

$$\begin{aligned} J(\mathbf{w}) &= \sum_{i=1}^n \left(\mathbf{y}^{(i)} - f(\mathbf{x}^{(i)}; \mathbf{w}) \right)^2 = \\ &= \sum_{i=1}^n \left(\mathbf{y}^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)} \right)^2 \end{aligned}$$

$$\mathbf{y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_d^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \cdots & x_d^{(n)} \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix}$$

$$J(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$$

Minimizing cost function

Optimal linear weight vector (for SSE cost function):

$$J(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$$

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = -2\mathbf{X}^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \mathbf{0} \Rightarrow \mathbf{X}^T \mathbf{X}\mathbf{w} = \mathbf{X}^T \mathbf{y}$$

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

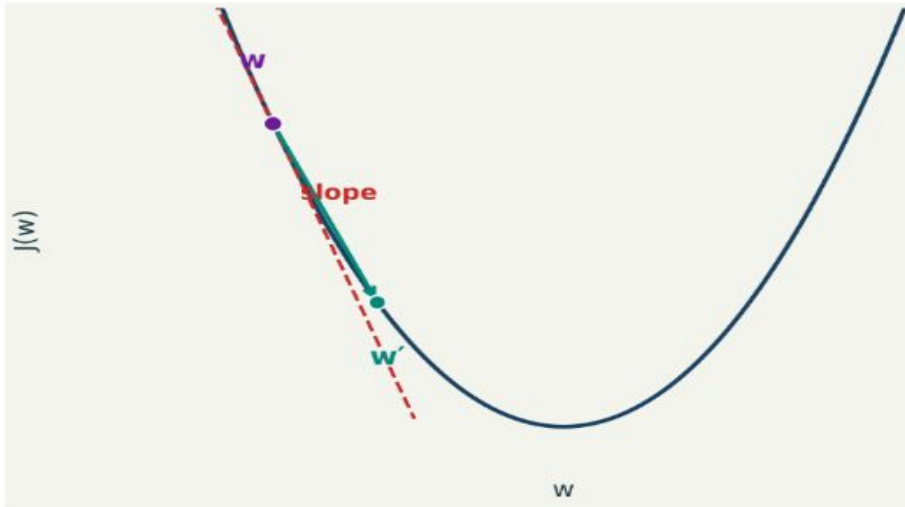
Computational limitation of analytical solution

- Analytical solutions do not scale well with very large datasets, making them impractical for big data applications
- Matrix may be near singular
- Only of MSE loss function
- Can not use for online learning

Another approach for optimizing SSE

- ▶ Iterative approach for solving the following optimization problem:

$$J(\mathbf{w}) = \sum_{i=1}^n (y^{(i)} - f(\mathbf{x}^{(i)}; \mathbf{w}))^2$$



Iterative optimization of cost function

- ▶ Cost function: $J(\mathbf{w})$
- ▶ Optimization problem: $\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} J(\mathbf{w})$
- ▶ Steps:
 - ▶ Start from $\mathbf{w}^0$
 - ▶ Repeat
 - ▶ Update $\mathbf{w}^t$ to $\mathbf{w}^{t+1}$ in order to reduce J
 - ▶ $t \leftarrow t + 1$
 - ▶ until we hopefully end up at a minimum

Gradient descent

- ▶ First-order optimization algorithm to find $\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} J(\mathbf{w})$
 - ▶ Also known as "**steepest descent**"
- ▶ In each step, takes steps proportional to the negative of the gradient vector of the function at the current point $\mathbf{w}^t$:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \gamma_t \nabla J(\mathbf{w}^t)$$

- ▶ $J(\mathbf{w})$ decreases fastest if one goes from $\mathbf{w}^t$ in the direction of $-\nabla J(\mathbf{w}^t)$
- ▶ Assumption: $J(\mathbf{w})$ is defined and differentiable in a neighborhood of a point $\mathbf{w}^t$

Gradient ascent takes steps proportional to (the positive of) the gradient to find a local maximum of the function

Gradient descent

- Minimize $J(\mathbf{w})$

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \nabla_{\mathbf{w}} J(\mathbf{w}^t)$$

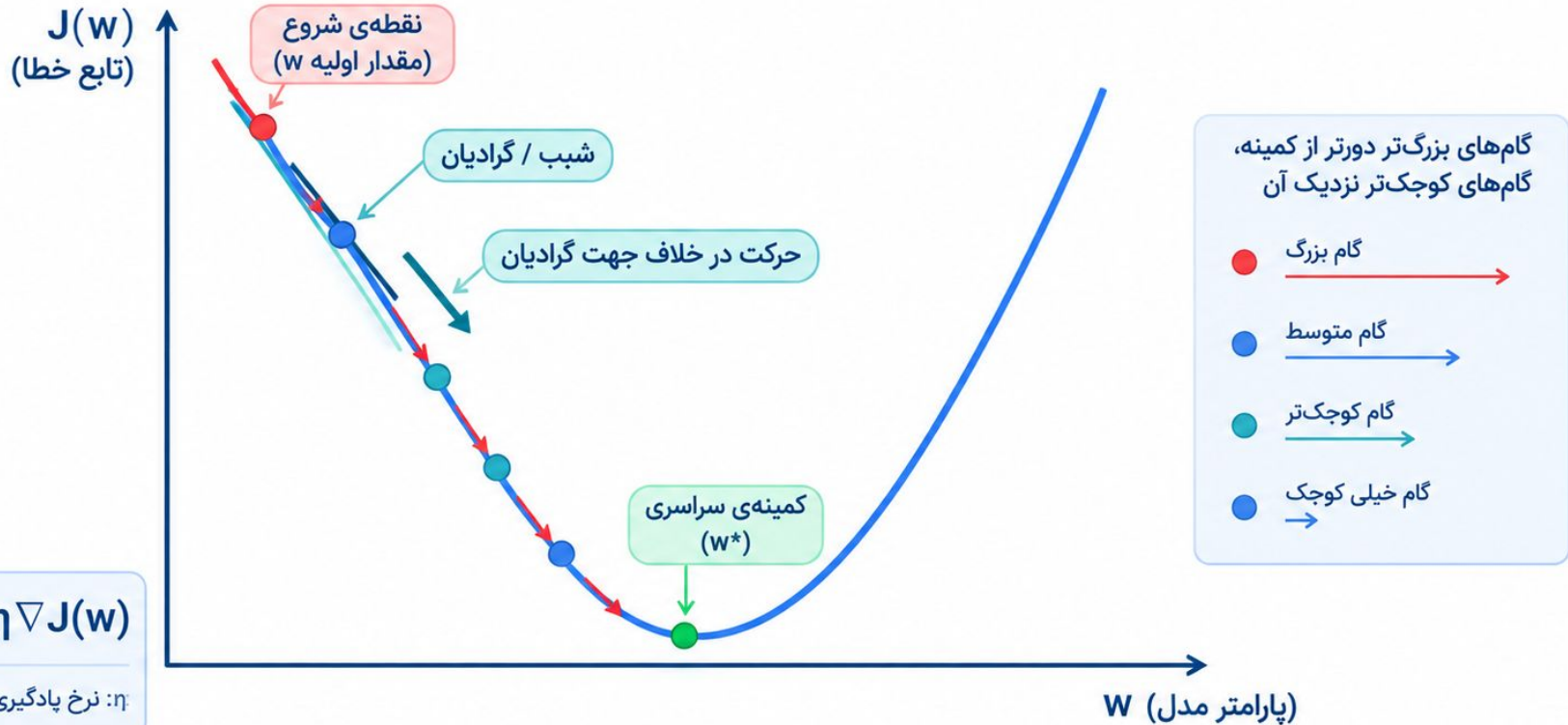
Step size
(Learning rate parameter)



$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \left[\frac{\partial J(\mathbf{w})}{\partial w_1}, \frac{\partial J(\mathbf{w})}{\partial w_2}, \dots, \frac{\partial J(\mathbf{w})}{\partial w_d} \right]$$

- If η is small enough, then $J(\mathbf{w}^{t+1}) \leq J(\mathbf{w}^t)$.
- η can be allowed to change at every iteration as η_t .

شهودگرادیان کاهشی (Gradient Descent)

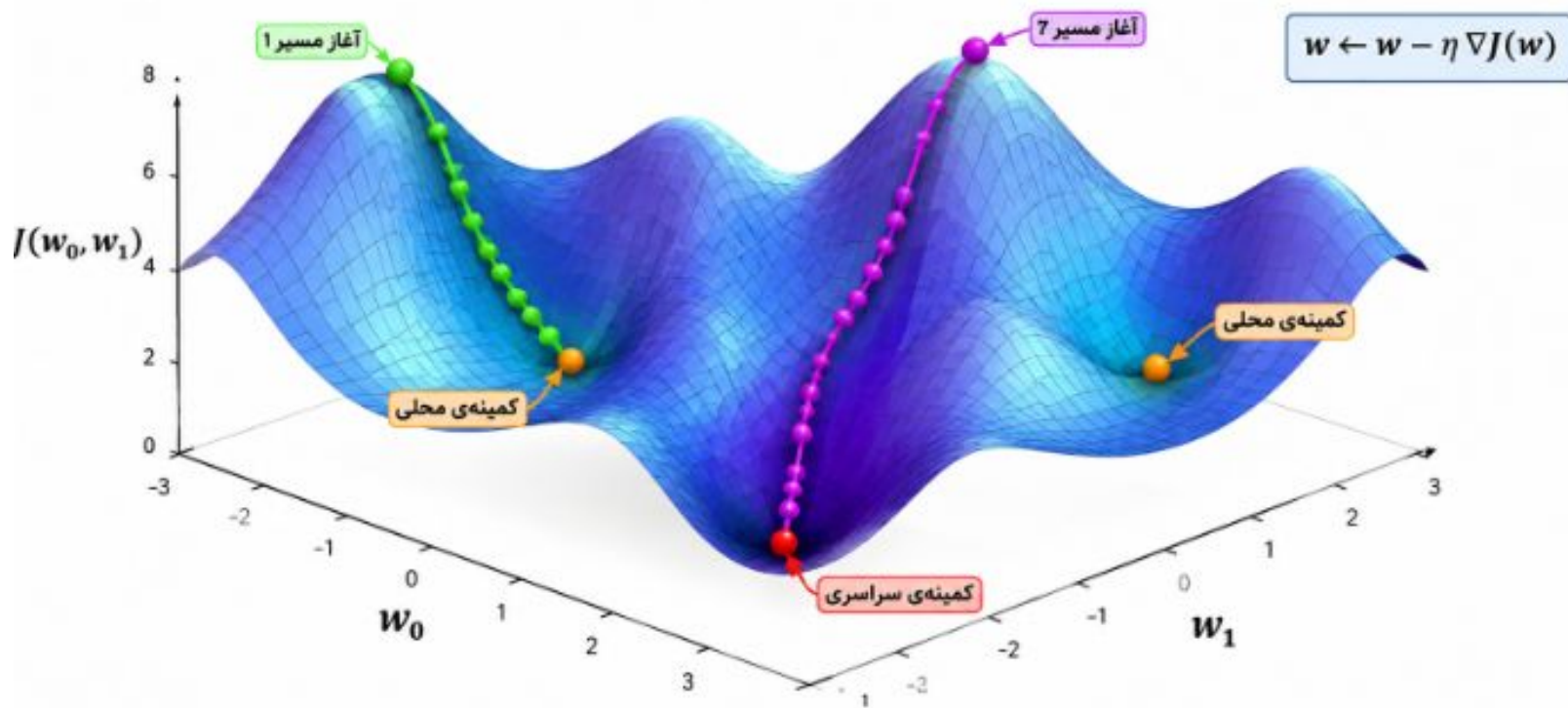


گرادیان‌کاهشی در هر گام، پارامتر را در خلاف جهت شیب تابع خطا حرکت می‌دهد تا به کمینه نزدیک‌تر شود.

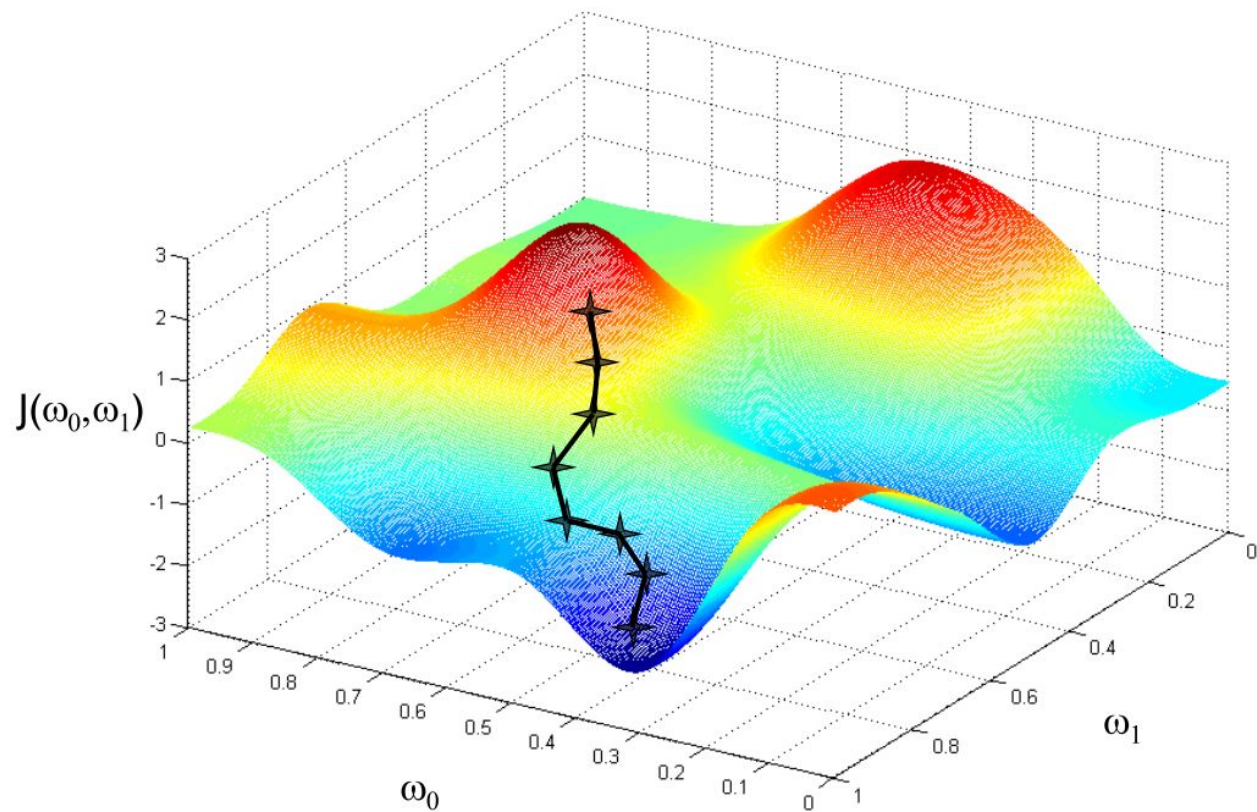
Gradient descent disadvantages

- ▶ Local minima problem
- ▶ However, when J is convex, all local minima are also global minima $\Rightarrow$ gradient descent can converge to the global solution.

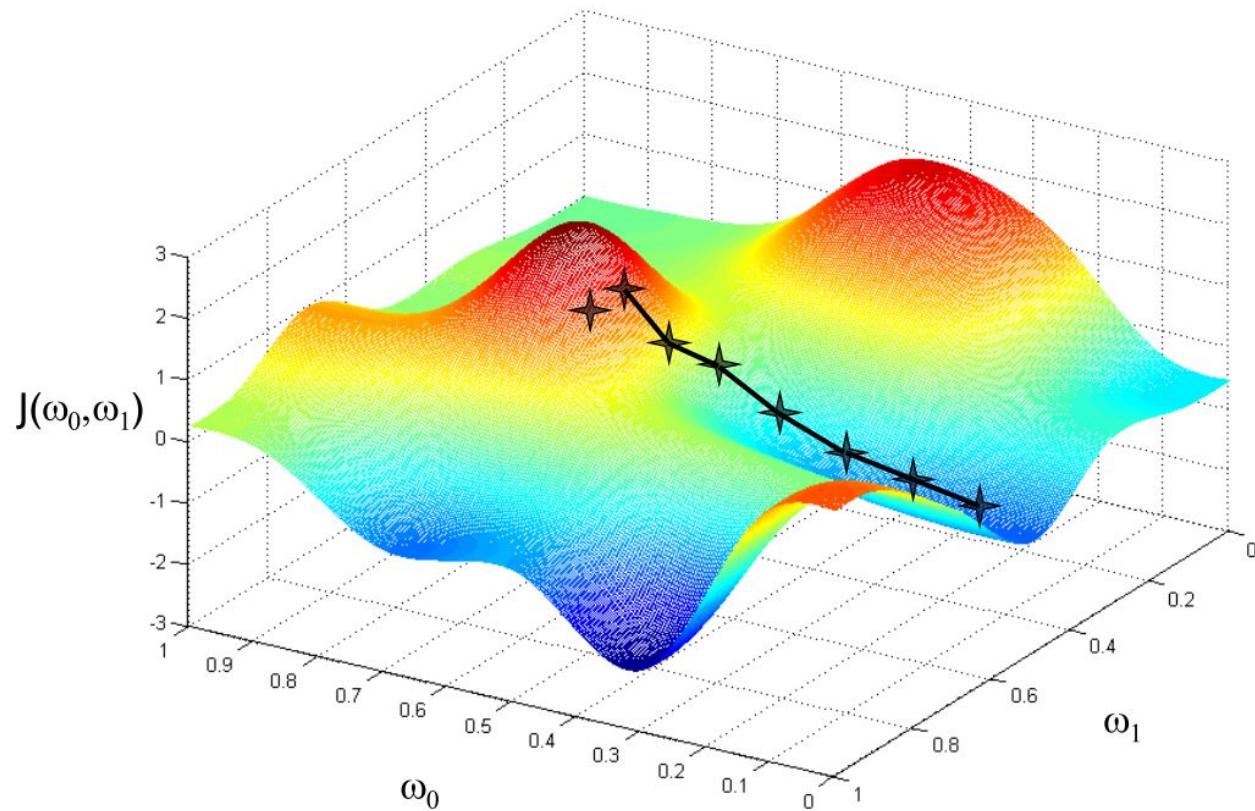
Gradient Descent Disadvantages



Review: Problem of gradient descent with non-convex cost functions



Review: Problem of gradient descent with non-convex cost functions



Gradient descent for SSE cost function

- ▶ Minimize $J(\mathbf{w})$

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \nabla_{\mathbf{w}} J(\mathbf{w}^t)$$

- ▶ $J(\mathbf{w})$: Sum of squares error

$$J(\mathbf{w}) = \sum_{i=1}^n \left(y^{(i)} - f(\mathbf{x}^{(i)}; \mathbf{w}) \right)^2$$

- ▶ Weight update rule for $f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T \mathbf{x}$:

$$\mathbf{w}^{t+1} = \mathbf{w}^t + \eta \sum_{i=1}^n \left(y^{(i)} - \mathbf{w}^{tT} \mathbf{x}^{(i)} \right) \mathbf{x}^{(i)}$$

Gradient descent for SSE cost function

- ▶ Weight update rule: $f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^T \mathbf{x}$

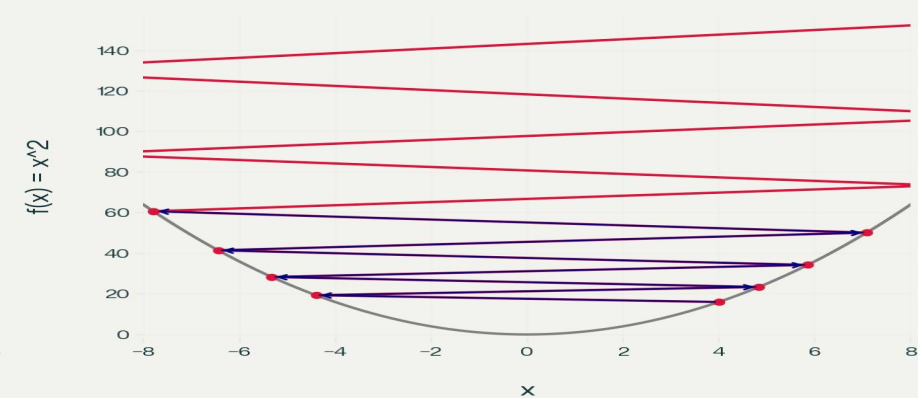
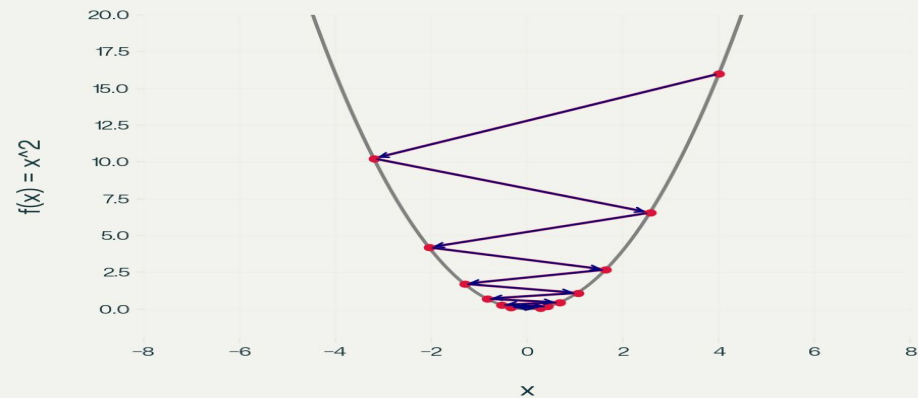
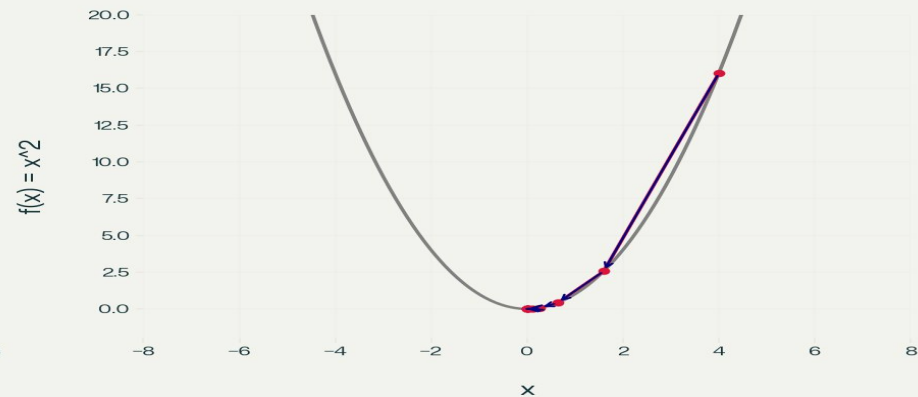
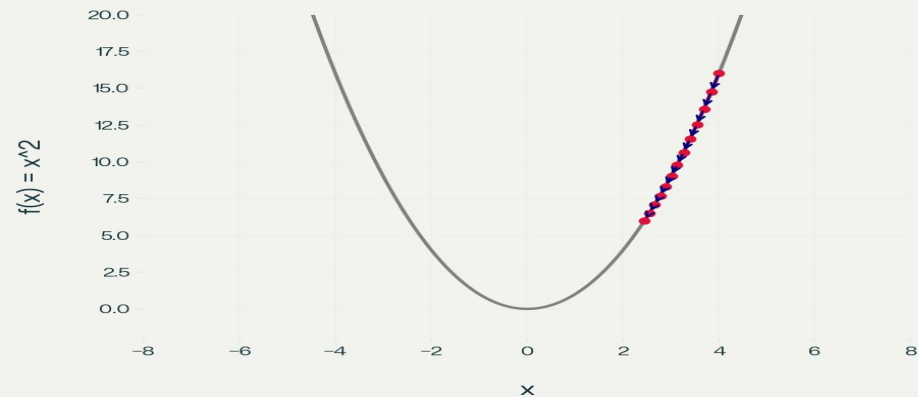
$$\mathbf{w}^{t+1} = \mathbf{w}^t + \eta \sum_{i=1}^n (y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)}) \mathbf{x}^{(i)}$$

Batch mode: each step
considers all training data

- ▶ η : too small $\rightarrow$ gradient descent can be slow.
- ▶ η : too large $\rightarrow$ gradient descent can overshoot the minimum. It may fail to converge, or even diverge.

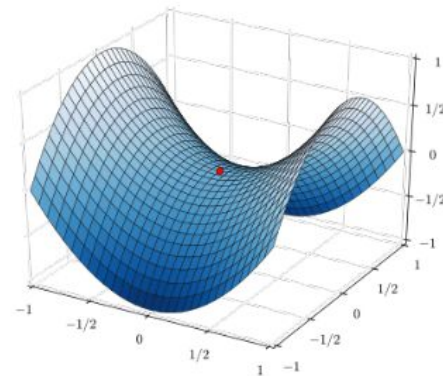
Effect of learning rate

Effect of learning rate on gradient descent convergence ($f(x) = x^2$, $x_0 = 4$)



Problem with GD

- High Variability (SGD): Quick in steep directions but slow in shallow ones, causing jitter and slow progress.
- Local Minima and Saddle Points: Risk of sub-optimal solutions or long convergence times in flat regions.
- Can not be used of online learning
 - Data observations are arriving in a continuous stream
 - Predictions must be made before seeing all of the data.
- Solution
 - Noisy Updates: Using individual points or mini-batches introduces noise, affecting stable convergence.



Stochastic gradient descent

- ▶ Batch techniques process the entire training set in one go
 - ▶ thus they can be computationally costly for large data sets.
- ▶ Stochastic gradient descent: when the cost function can comprise a sum over data points:

$$J(\mathbf{w}) = \sum_{i=1}^n J^{(i)}(\mathbf{w})$$

- ▶ Update after presentation of $(\mathbf{x}^{(i)}, y^{(i)})$:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \nabla_{\mathbf{w}} J^{(i)}(\mathbf{w})$$

Variants of GD

- **Batch gradient descent:** processes the entire training set in one iteration
 - It can be computationally costly for large data sets and practically impossible for some applications (such as online learning).
- **Mini-batch gradient descent:** processes small, random subsets (mini-batches) of the training set in each iteration
 - Balances the efficiency of batch gradient descent.
- **Stochastic gradient descent:** processes one training example per iteration
 - Updates the model parameters more frequently, which can lead to faster convergence.

Stochastic gradient descent

- ▶ Example: Linear regression with SSE cost function

$$J^{(i)}(\mathbf{w}) = (y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2$$

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \nabla_{\mathbf{w}} J^{(i)}(\mathbf{w})$$

$$\mathbf{w}^{t+1} = \mathbf{w}^t + \eta (y^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)}) \mathbf{x}^{(i)}$$

Least Mean Squares (LMS)

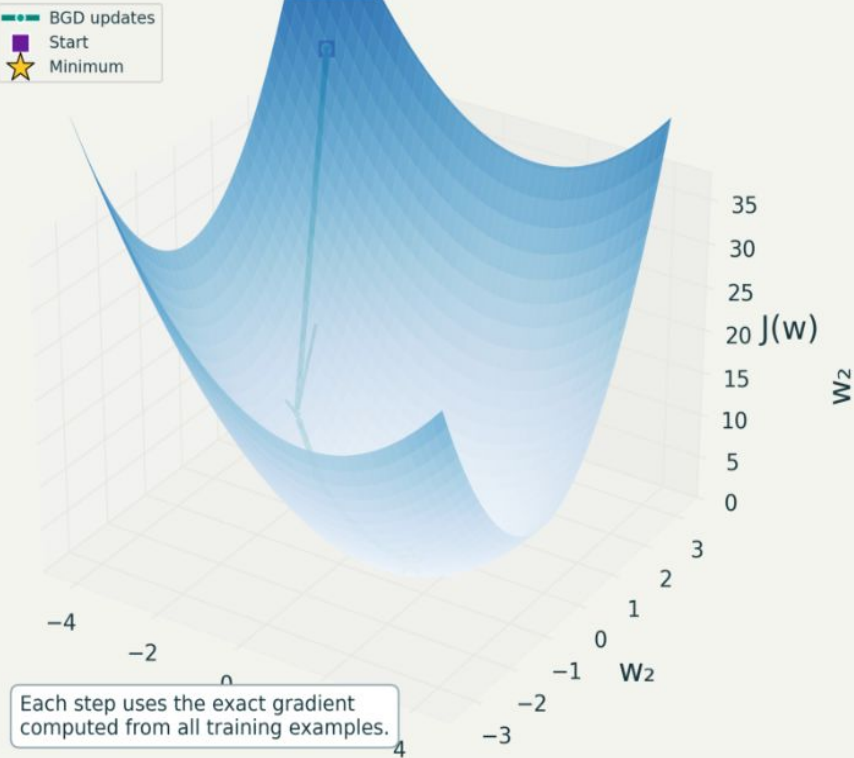
It is proper for sequential or online learning

Stochastic gradient descent: online learning

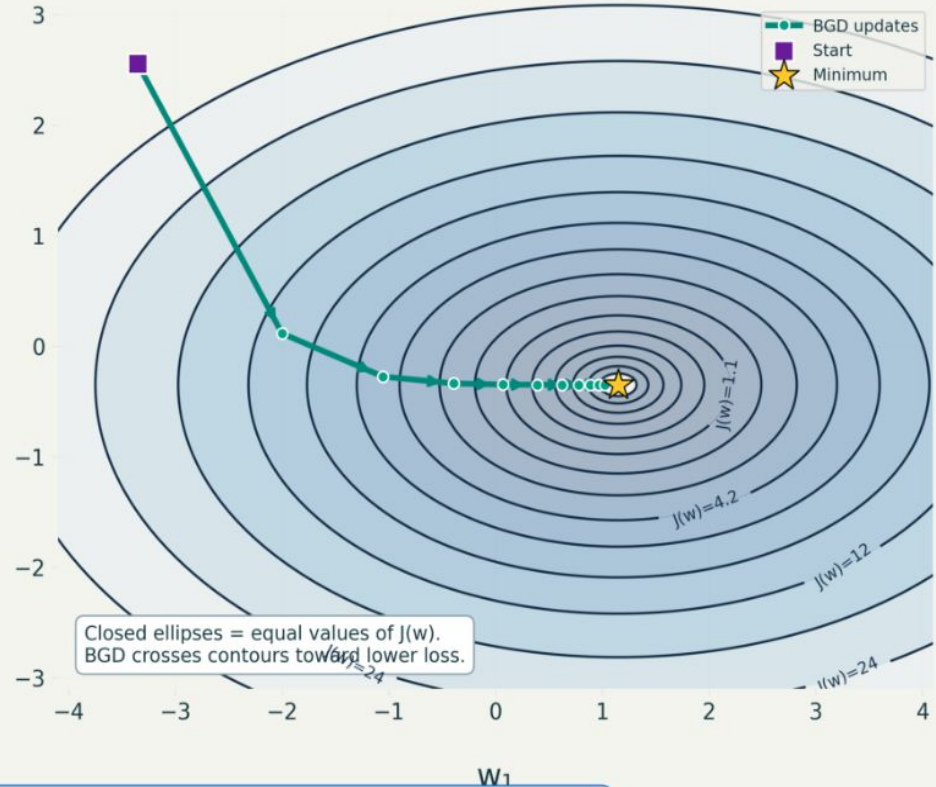
- ▶ Sequential learning is also appropriate for real-time applications
 - ▶ data observations are arriving in a continuous stream
 - ▶ and predictions must be made before seeing all of the data
- ▶ The value of η needs to be chosen with care to ensure that the algorithm converges

Batch Gradient Descent (BGD): Minimizing the Loss Function $J(w)$

3D loss surface $J(w_1, w_2)$



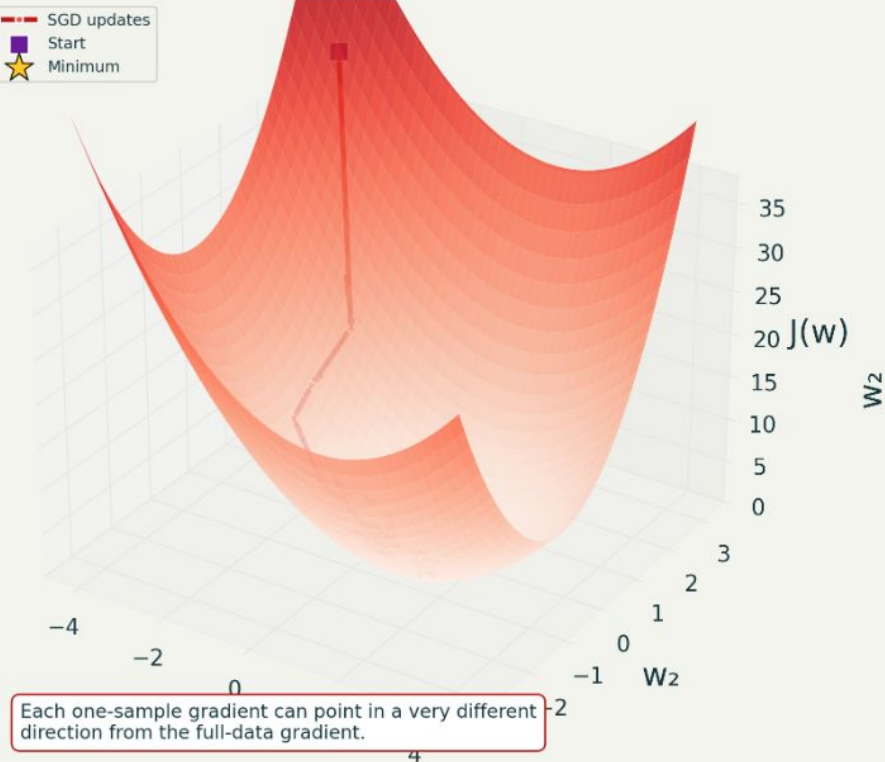
Contour map of $J(w_1, w_2)$



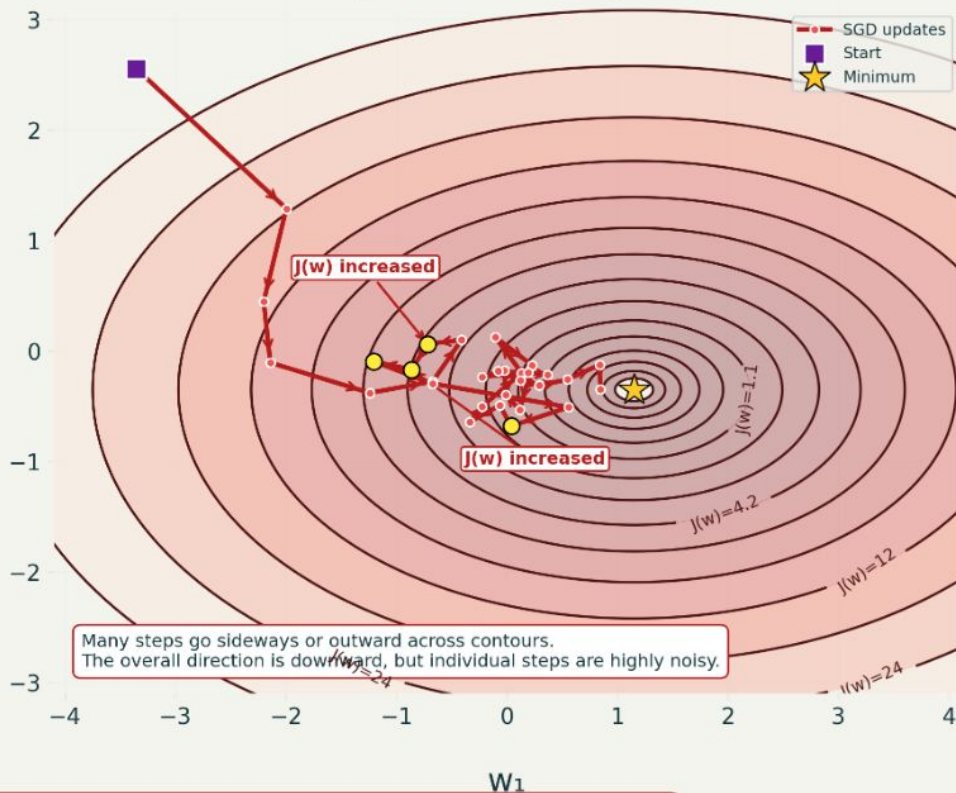
BGD update: $w_{t+1} = w_t - \eta \nabla J(w_t)$ | ∇J is computed using ALL training data in every update

Stochastic Gradient Descent (SGD): Highly Noisy, Zig-Zag Updates

3D loss surface $J(w_1, w_2)$



Contour map: much stronger SGD noise



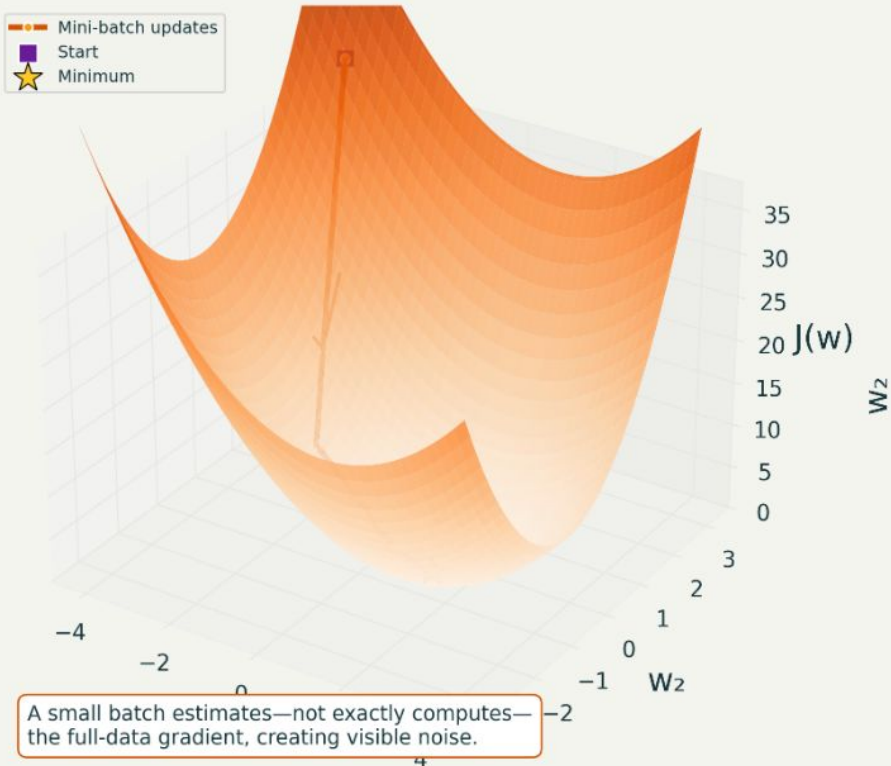
SGD: $w_{t+1} = w_t - \eta \nabla L_{it}(w_t)$ | One example per update $\rightarrow$ high-variance, strongly noisy gradient estimates



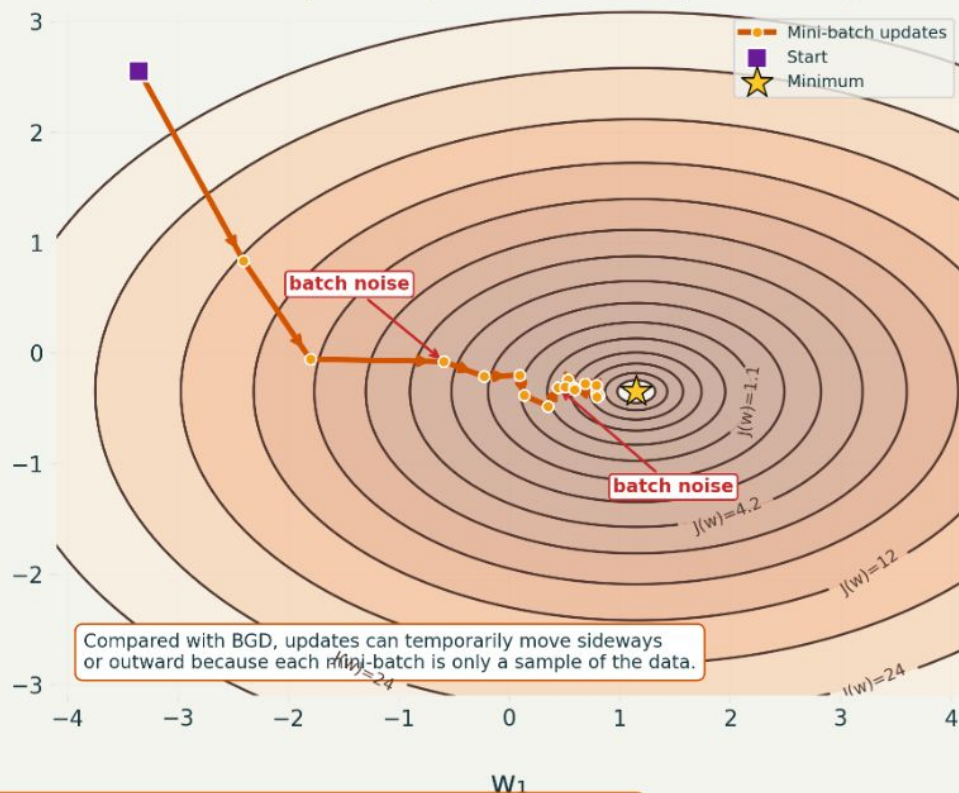
Mini-batch Gradient Descent: Visible Gradient Noise from Small Batches



3D loss surface $J(w_1, w_2)$



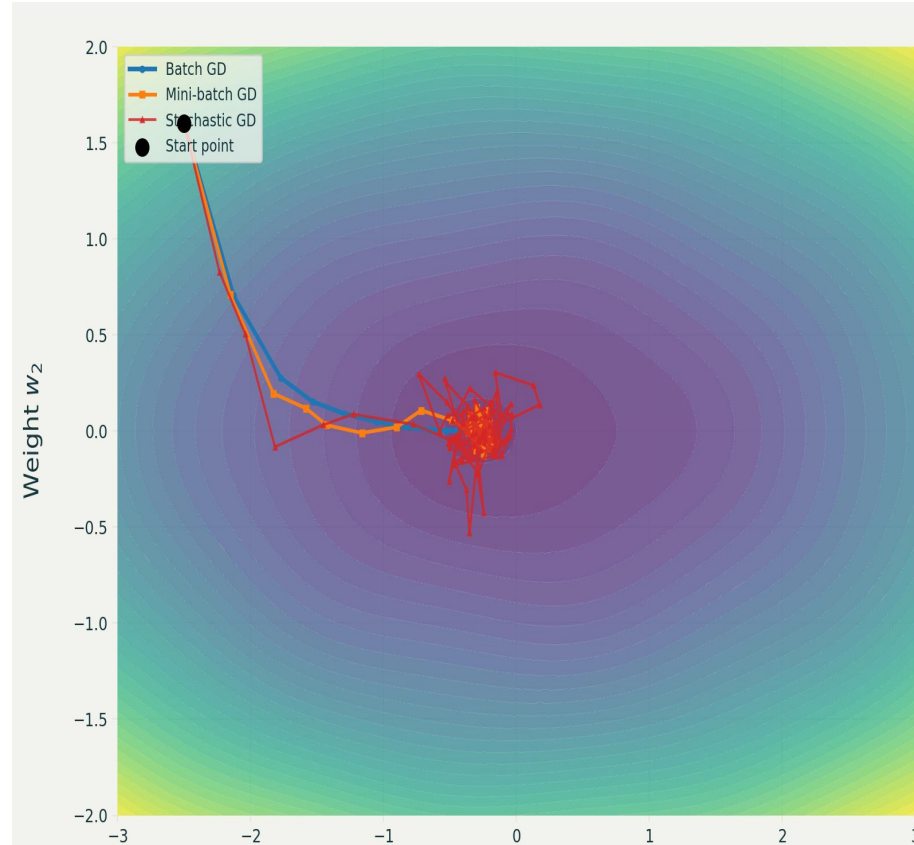
Contour map: noisy but generally inward path



Mini-batch GD: $w_{t+1} = w_t - \eta (1/|B_t|) \sum_{i \in B_t} \nabla L_i(w_t)$ | Smaller batches $\rightarrow$ noisier gradient estimates

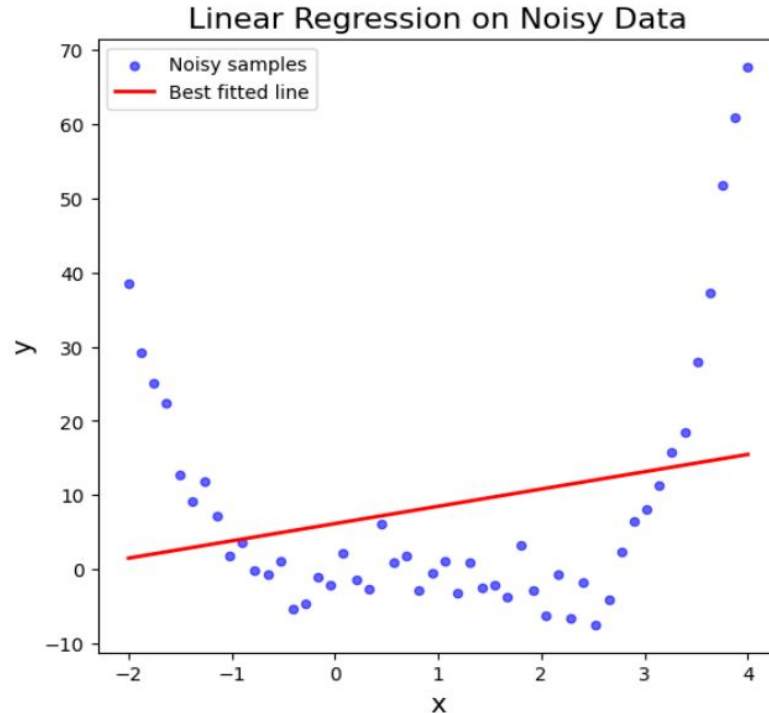
Batch GD- mini Batch GD - SGD

- Often, stochastic gradient descent gets close to the minimum much faster than batch gradient descent.
- Note however that it may never converge to the minimum, and the parameters will keep oscillating around the minimum of the cost function;



Limitations of linear regression

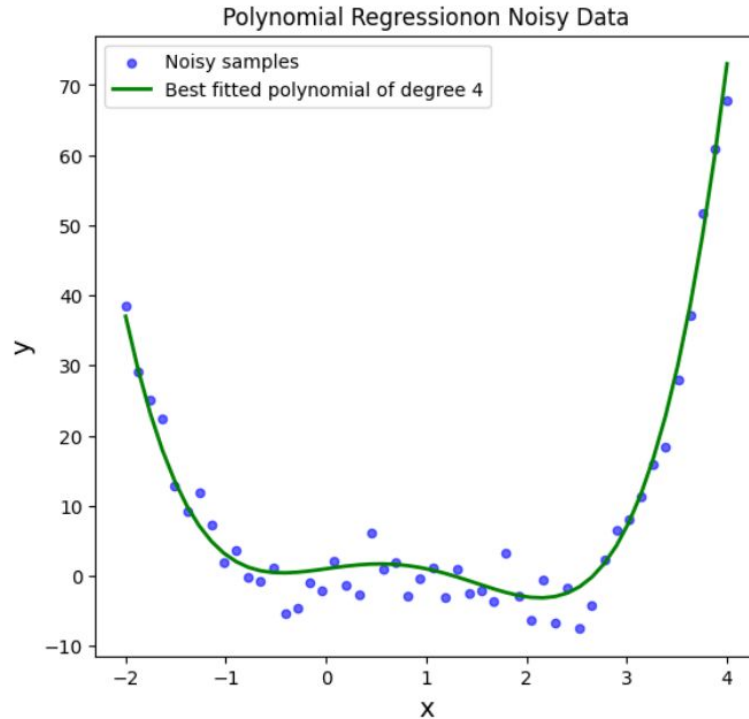
- It is possible that the best fitted line is still far off the real pattern of samples:



Beyond linear regression

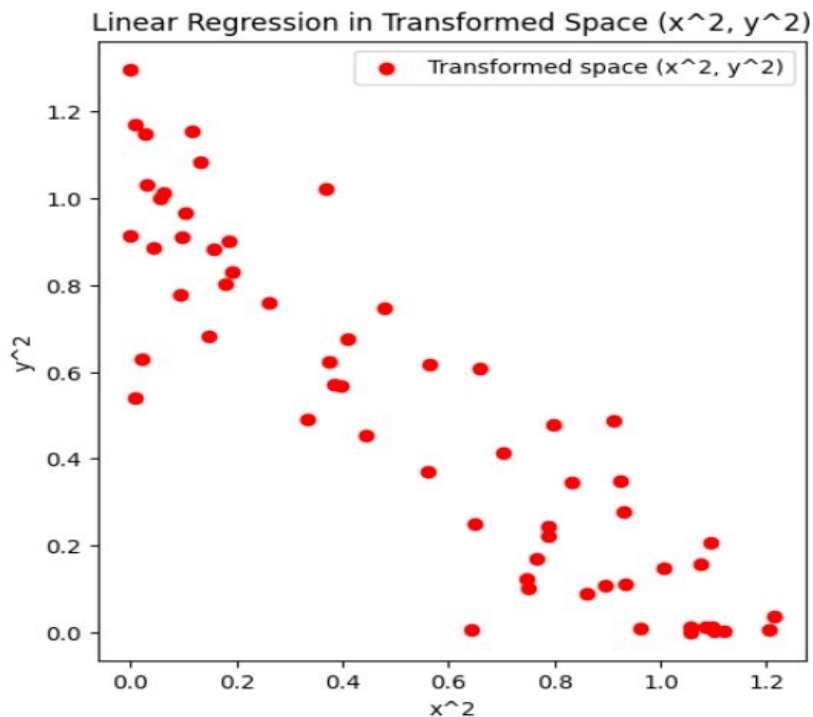
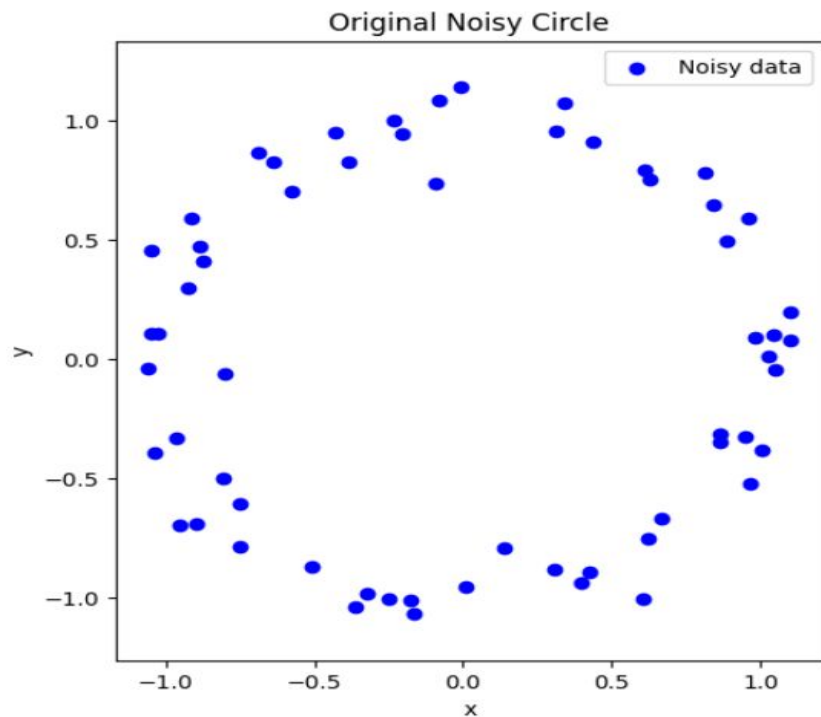
- How can we extend linear regression to model non-linear relationships?
 - **Transforming Data Using Basis Functions:**
 - Basis functions allow us to transform the original features into a new feature space.
 - Common basis functions include polynomial and Gaussian functions.
 - **Learning a Linear Regression on Transformed Features:**
 - By applying linear regression to the transformed feature vectors, we can model complex, non-linear relationships.
 - This approach maintains the simplicity and interpretability of linear regression while extending its flexibility.

Polynomial regression



Best fitted polynomial of degree 4 can generalize well on samples

Change of basis



Changing the basis of $[1, x, y]$ to $[1, x^2, y^2]$, we can use linear regression

Polynomial regression : univariate

- Similar to what we did for univariate linear regression, we can define:

$$\mathbf{X}' = \begin{bmatrix} 1 & x^{(1)1} & \dots & x^{(1)m} \\ 1 & x^{(2)1} & \dots & x^{(2)m} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x^{(n)1} & \dots & x^{(n)m} \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} \hat{w}_0 \\ \hat{w}_1 \\ \vdots \\ \hat{w}_m \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

In which $x^{(i)}$ indicates the i -th data point.

Generalized linear

- ▶ Linear combination of fixed non-linear function of the input vector

$$f(\mathbf{x}; \mathbf{w}) = w_0 + w_1 \phi_1(\mathbf{x}) + \dots w_m \phi_m(\mathbf{x})$$

$\{\phi_1(\mathbf{x}), \dots, \phi_m(\mathbf{x})\}$: set of basis functions (or features)

$$\phi_i(\mathbf{x}): \mathbb{R}^d \rightarrow \mathbb{R}$$

Basis functions : example

Linear

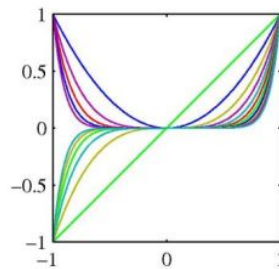
If $m = d$, $\phi_i(\mathbf{x}) = x_i$, $i = 1, \dots, d$, then

$$f(\mathbf{x}; \mathbf{w}) = w_0 + w_1x_1 + \dots + w_dx_d$$

Polynomial (univariate)

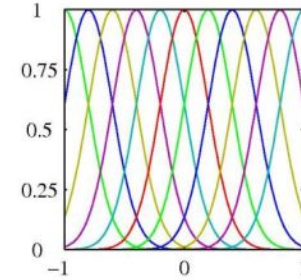
If $\phi_i(x) = x^i$, $i = 1, \dots, m$, then

$$f(x; \mathbf{w}) = w_0 + w_1x + \dots + w_{m-1}x^{m-1} + w_mx^m$$



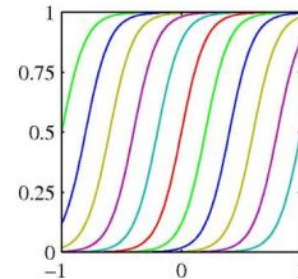
Basis functions : example

Gaussian: $\phi_j(\mathbf{x}) = \exp \left\{ -\frac{(\mathbf{x} - \mathbf{c}_j)^2}{2\sigma_j^2} \right\}$



Sigmoid: $\phi_j(\mathbf{x}) = \sigma \left(\frac{\|\mathbf{x} - \mathbf{c}_j\|}{\sigma_j} \right)$

$$\sigma(a) = \frac{1}{1 + \exp(-a)}$$



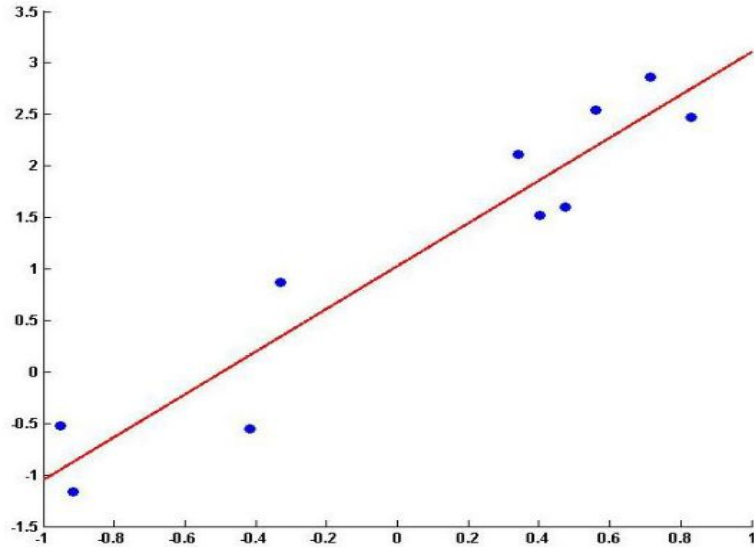
Generalized linear : Optimization

$$\begin{aligned} J(\mathbf{w}) &= \sum_{i=1}^n \left(y^{(i)} - f(\mathbf{x}^{(i)}; \mathbf{w}) \right)^2 \\ &= \sum_{i=1}^n \left(y^{(i)} - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}^{(i)}) \right)^2 \end{aligned}$$

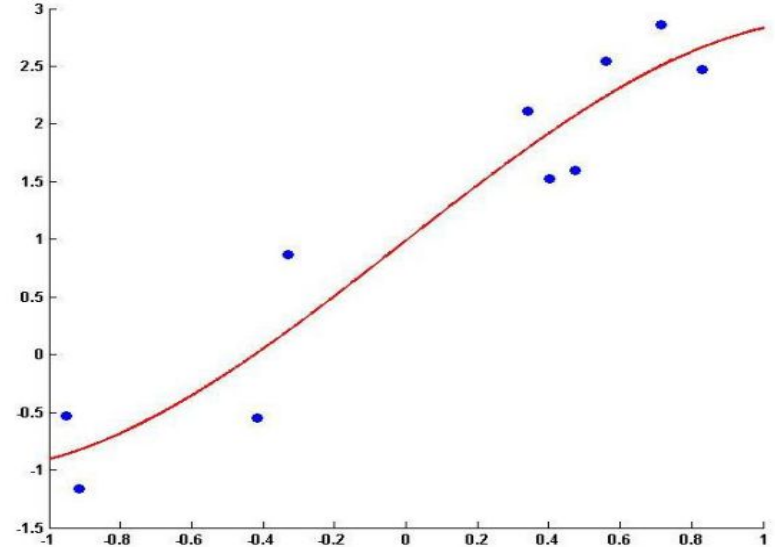
$$\mathbf{y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix} \quad \boldsymbol{\Phi} = \begin{bmatrix} 1 & \phi_1(\mathbf{x}^{(1)}) & \cdots & \phi_m(\mathbf{x}^{(1)}) \\ 1 & \phi_1(\mathbf{x}^{(2)}) & \cdots & \phi_m(\mathbf{x}^{(2)}) \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \phi_1(\mathbf{x}^{(n)}) & \cdots & \phi_m(\mathbf{x}^{(n)}) \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_m \end{bmatrix}$$

$$\widehat{\mathbf{w}} = (\boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T \mathbf{y}$$

Which degree is better

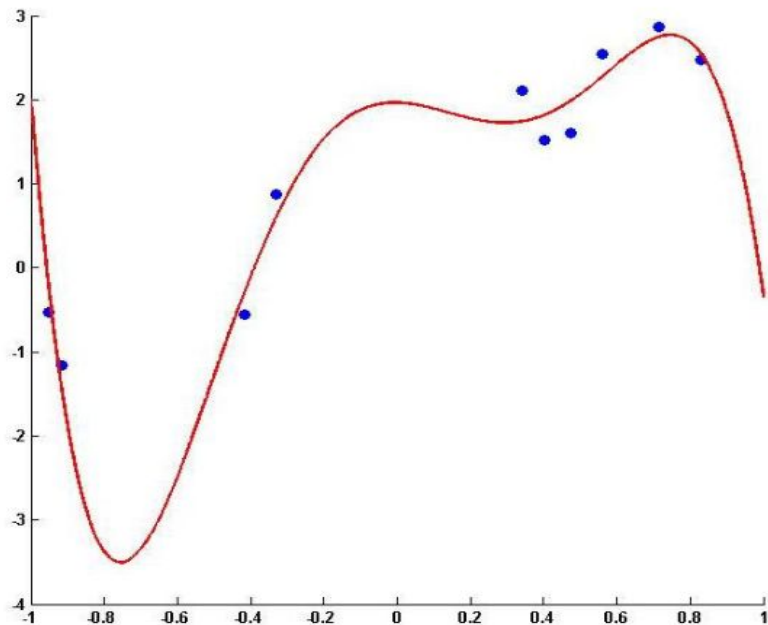


Degree of 1

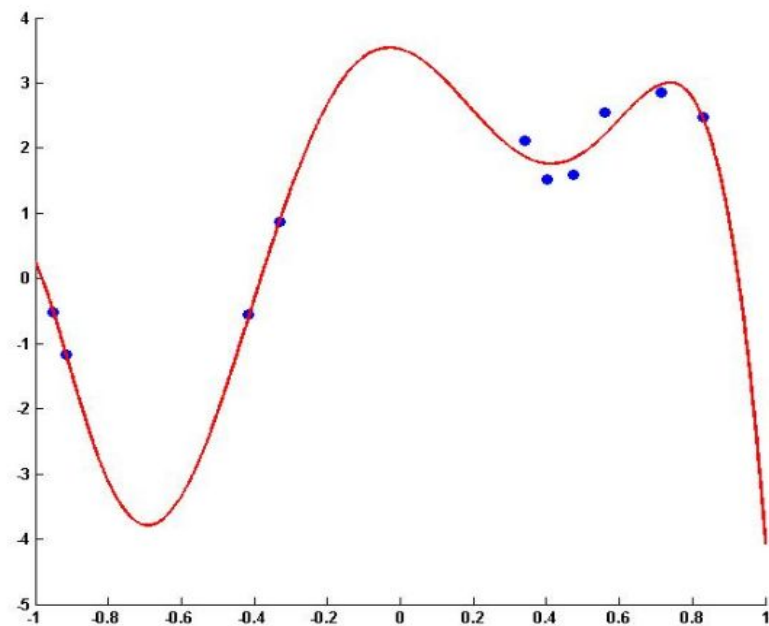


Degree of 3

Overfitting



Degree of 5



Degree of 7

Reference

Soleimani, M . *Machine Learning Course Lecture Slides*.

James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). An Introduction to Statistical Learning: With Applications in Python. Springer.

Sharifi zarchi, A., Machine learning Course Lecture Slides.